香港中文大學統計學系
Department of Statistics
THE CHINESE UNIVERSITY OF HONG KONG

STAT6207
Applied Deep Learning
L2, 18 Sep 2026
By Dr. Ming Cheung



Announcement

- Midterm exam (papers + programming) will be on 23 Oct 26
- Location: TBC
- There will be two parts
  - Written: MC and short question
  - Programming: You can use your computer and agent coder
- No communication is allowed

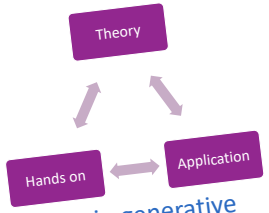


Lecture Room

Lecture	Week	Lecture Rooms
L1	11-Sept	BMS G18
L2	18-Sept	BMS G18
L3	25-Sept	CYT LT1A
L4	2-Oct	CYT LT1A
L5	9-Oct	CYT LT1A
L6	16-Oct	BMS G18
L7 (Mid Term)	23-Oct	To be confirmed
L8	30-Oct	CYT LT1A
L9	6-Nov	UCC C1
L10	13-Nov	CYT LT1A
L11	20-Nov	UCC C1
L12	27-Nov	UCC C1
L13	4-Dec	UCC C1
L14	11-Dec	UCC C1

Last Lecture

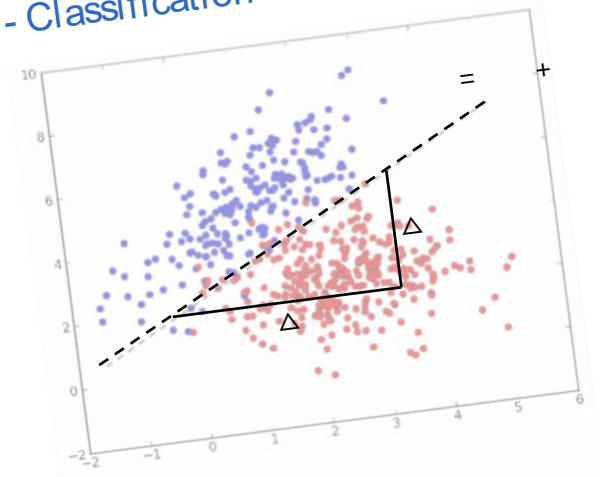
What is STAT6207



1. Learn the **theory** of machine learning and deep learning;
2. Understand machine learning **applications** to solve real problems in generative adversarial networks, large language model, etc.;
3. Gain **hands on** experience through projects and activities using Python libraries, such as PyTorch and Numpy, with projects

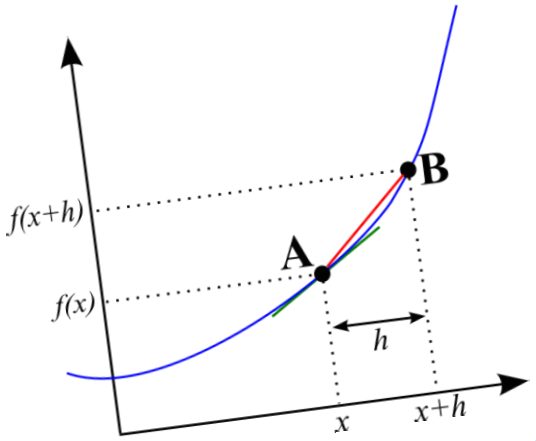
What is Machine Learning - Classification

- Using a straight line equation as the separator.
- Variables: and
- Constants: and
- On graph:
  - = gradient
  - = y-axis intercept



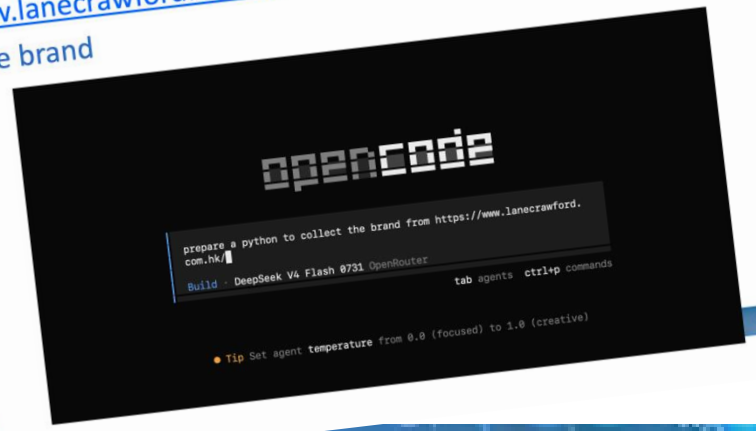
Training – following the slope

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



Demo: Scraping with OpenCode

- prepare a python to collect the brand from https://www.lanecrawford.com.hk/
- Visualize the brand



Agenda

- Linear classifier
- Transfer Learning
- Local Large Language Model



Linear Algebra

- Matrix addition (**A** + **B** = **C**)

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

```
1  import numpy as np
2
3  a = np.array([[1,2],[3,4]])
4  b = np.array([[5,6],[7,8]])
5
6  c = a + b
7
8  d = a * b
```

- Matrix element-wise multiplication (Hadamard product)

$$\begin{pmatrix} a_1 \\ a_2 \end{pmatrix} .* \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} a_1 b_1 \\ a_2 b_2 \end{pmatrix}$$

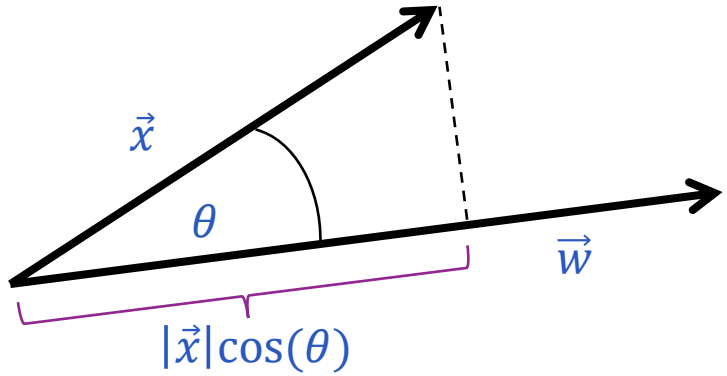


Linear Algebra

- Inner product: measuring the similarity of 2 vectors

$$(w_1, w_2, \dots, w_N)\begin{pmatrix} x_1 \\ \cdots \\ x_N \end{pmatrix} = w_1x_1 + w_2x_2 + \dots + w_Nx_N = \sum_{i=1}^{N} w_ix_i$$

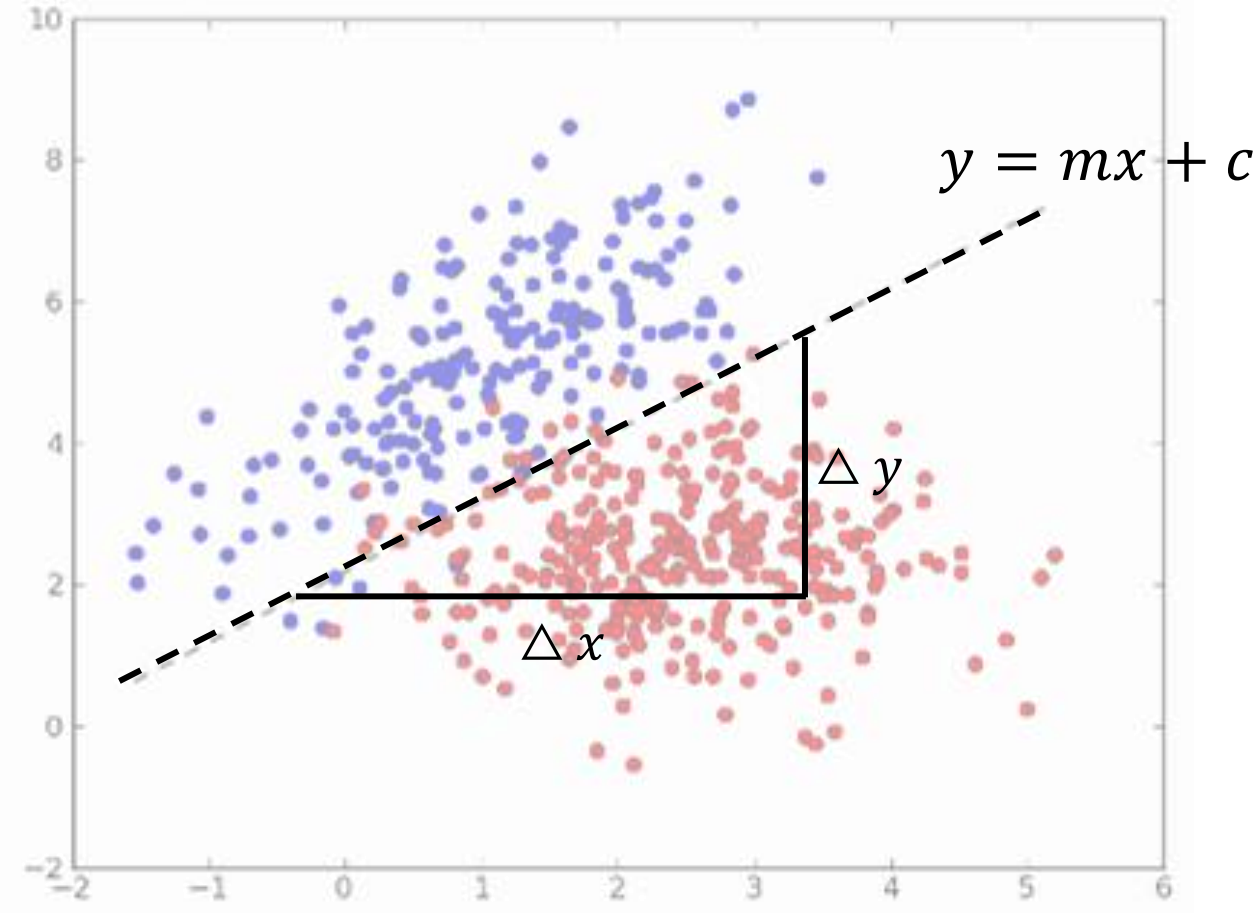
```
a = np.array([1,2,3,4])
b = np.array([0,5,1,6])
e=np.inner(a,b)
```



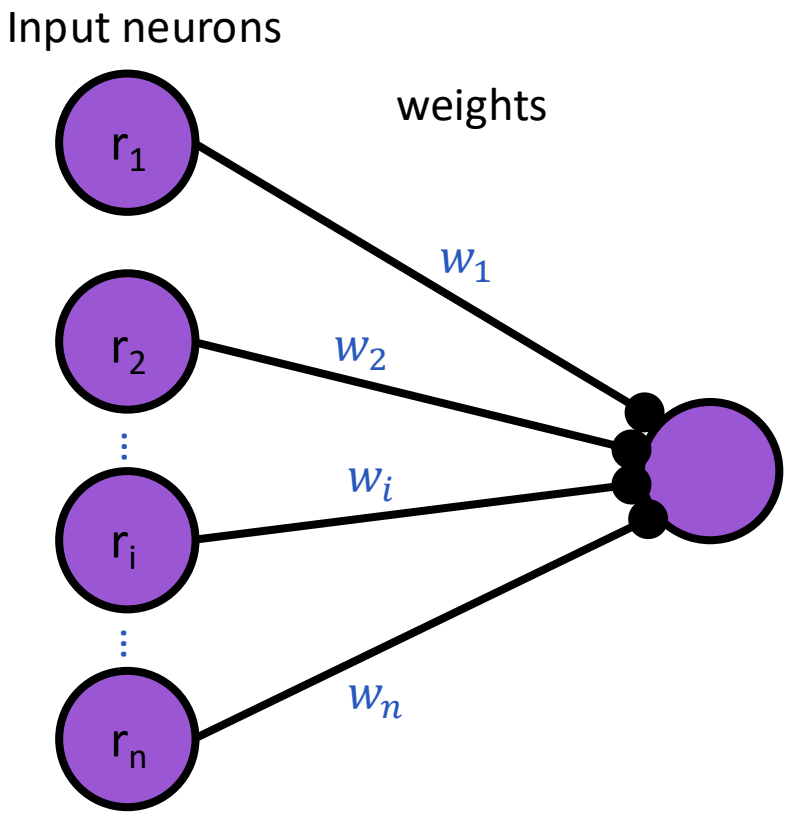
$$inner(\vec{x}, \vec{w}) = |\vec{x}||\vec{w}|\cos(\theta)$$

Recall: What is Machine Learning - Classification

- Using a straight line equation as the separator.
- Variables: x and y
- Constants: m and c
- On graph:
  - m = gradient
  - c = y-axis intercept
- How's about higher dimension?



Linear Algebra



$$(w_1 \quad w_2 \quad \cdots \quad w_N)\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} = w_1x_1 + w_2x_2 + \cdots + w_Nx_N$$



Examples of Linear Classification

- Training set $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, where $y_i \in [0,1]$
- The defined model:

$$p(y = 1|\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T\mathbf{x})}$$

$$p(y = 0|\mathbf{x}, \mathbf{w}) = 1 - \sigma(\mathbf{w}^T\mathbf{x}) = \frac{\exp(-\mathbf{w}^T\mathbf{x})}{1 + \exp(-\mathbf{w}^T\mathbf{x})}$$

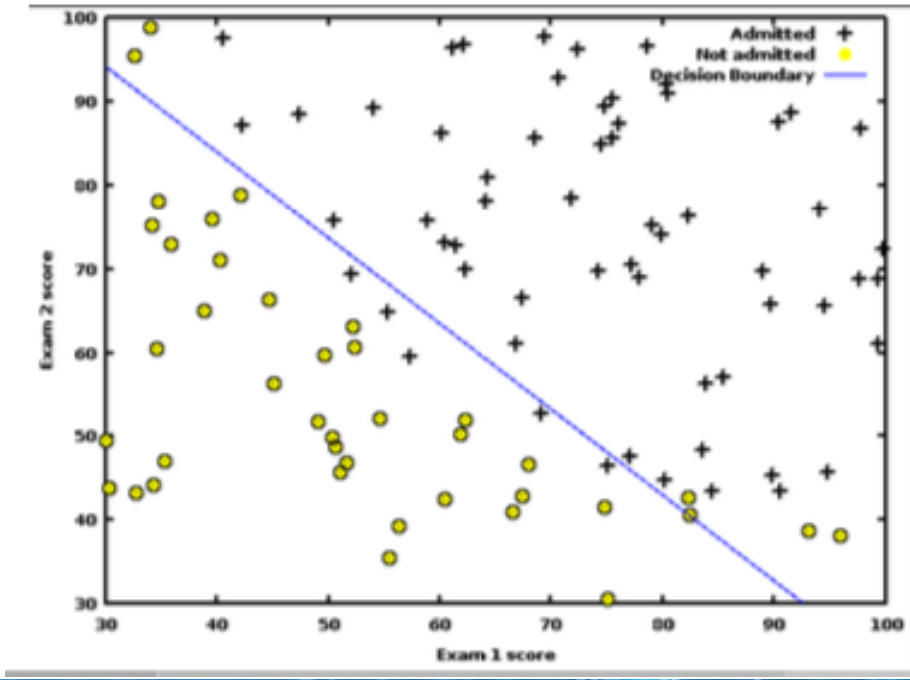
- Consider the **logit** of $p(y = 1|\mathbf{x}, \mathbf{w})$

$$\log \frac{p(y = 1|\mathbf{x}, \mathbf{w})}{1 - p(y = 1|\mathbf{x}, \mathbf{w})} = \log \frac{p(y = 1|\mathbf{x}, \mathbf{w})}{p(y = 0|\mathbf{x}, \mathbf{w})} = \log \exp(\mathbf{w}^T\mathbf{x}) = \mathbf{w}^T\mathbf{x}$$



Classification: Decision Rule

- To Classify instances, we obtain a point estimation of y:
  - $\hat{y}$=argmax $p(y|x,w)$
- In other words, the rules is: $\hat{y}$=1 iff $p(y=1|x,w)>0$
- This is called the optimal Bayes classifier
  - e.g., $\hat{y}$=1 iff $w^T x>0$ as a linear classifier





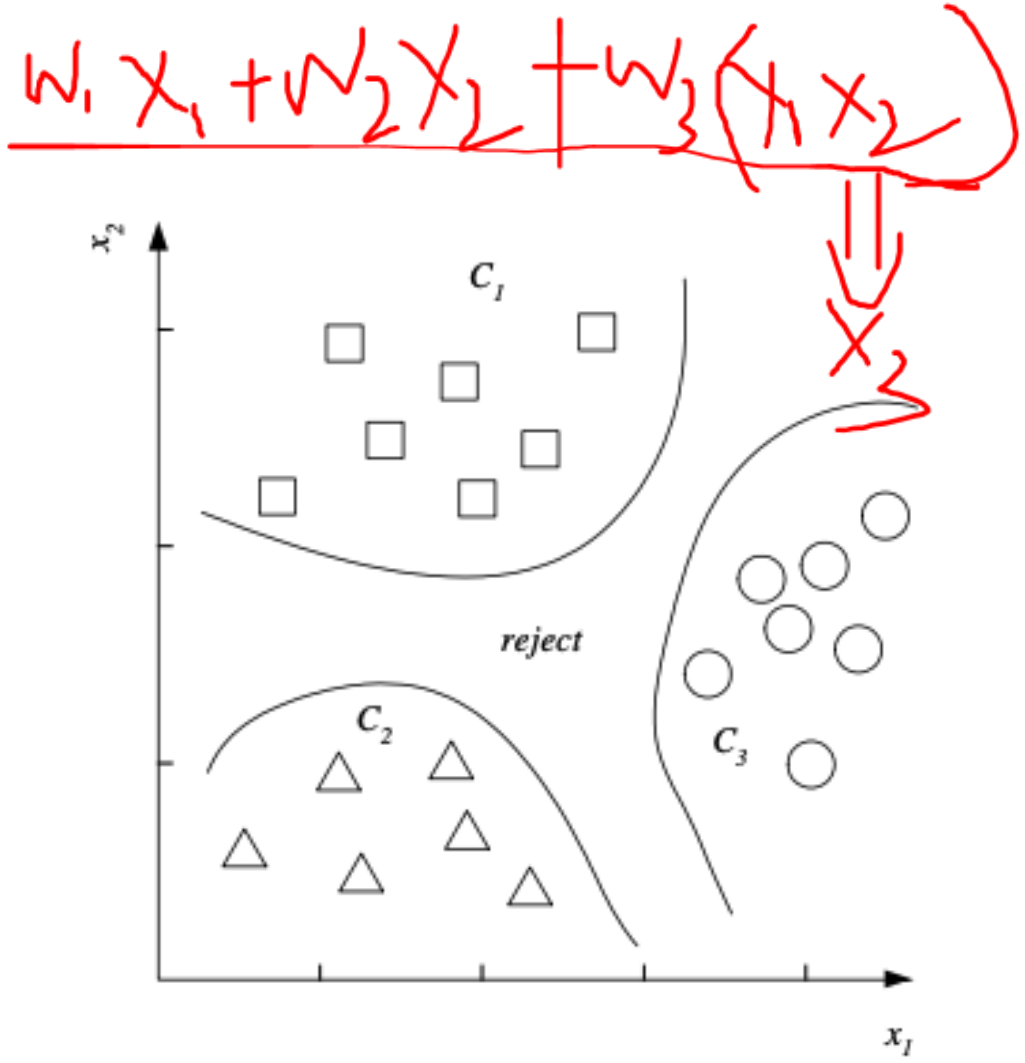
Decision Regions for Multi-classes

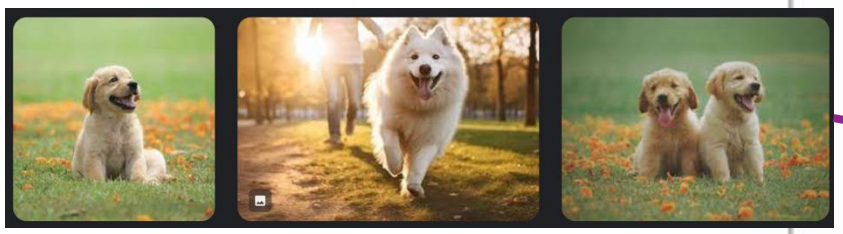
- The feature space is divided into K decision regions $\mathcal{R}_1, \ldots, \mathcal{R}_K$, where

$$\mathcal{R}_i = \left\{ \mathbf{x} \middle| g_i(\mathbf{x}) = \max_k g_k(\mathbf{x}) \right\}$$

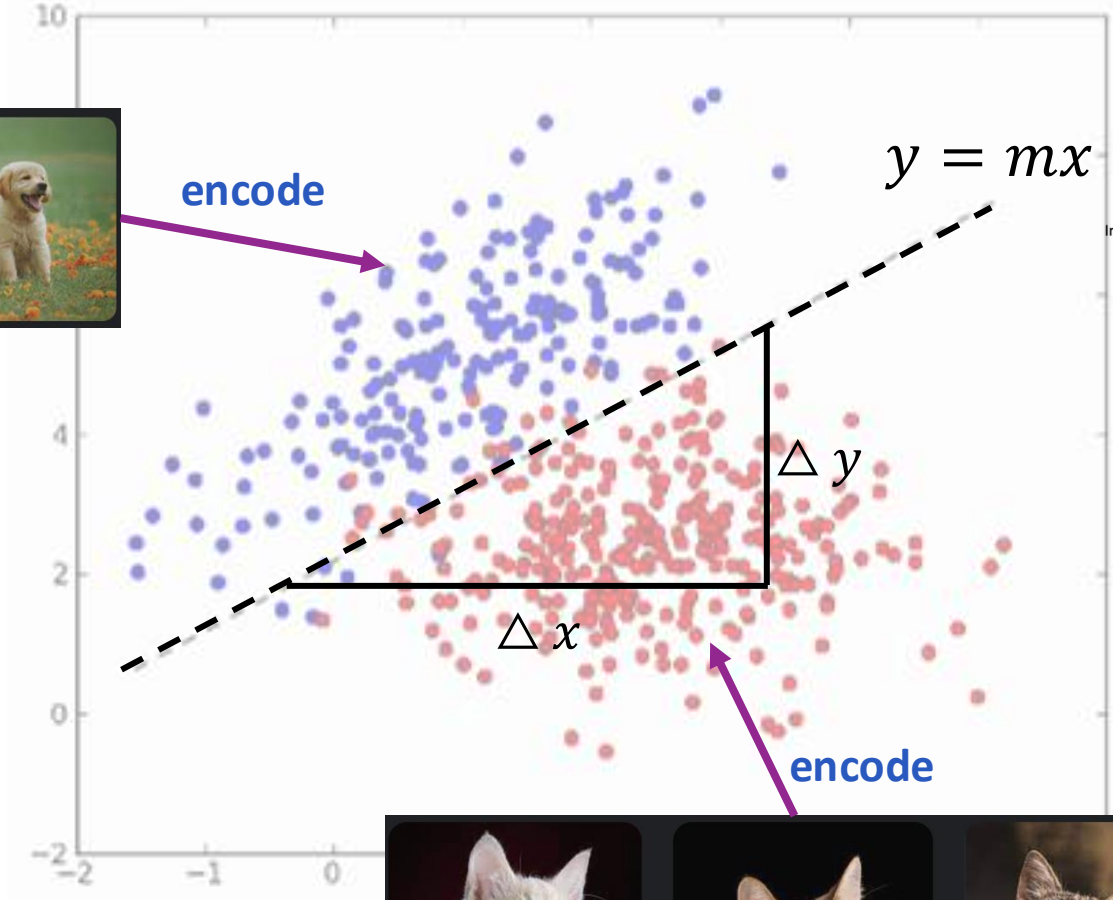
- The decision regions are separated by decision boundaries where ties occur among the largest discriminant functions.

- Choose C_i if $g_i(\mathbf{x}) = \max_k g_k(\mathbf{x})$

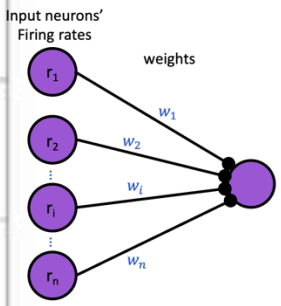


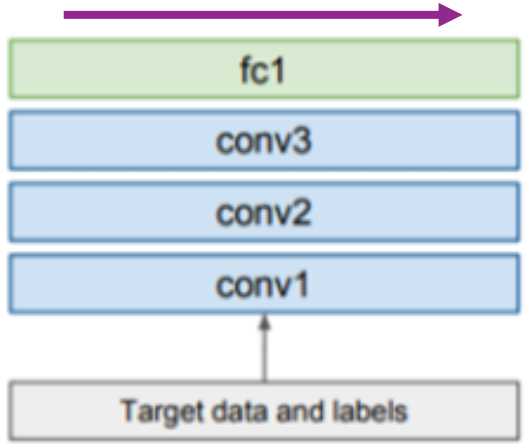


encode



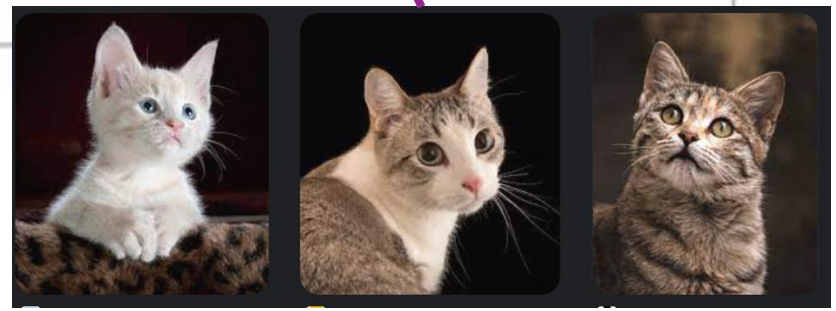
classifier





Pre-train model (encoder)

encode





RGB



Array of **32x32x3** numbers
(3072 numbers total)

	Red	Green	Blue	Hexadecimal code
	0	0	0	#000000
	255	255	255	#FFFFFF
	255	0	0	#FF0000
	0	255	0	#00FF00
	0	0	255	#0000FF
	255	128	0	#FF8000
	255	255	0	#FFFF00
	128	128	128	#808080

Image Classification with Linear Classifier

Training is to find **W** such that $f(x,\mathbf{W})$ gives the desired output, for an input image, x

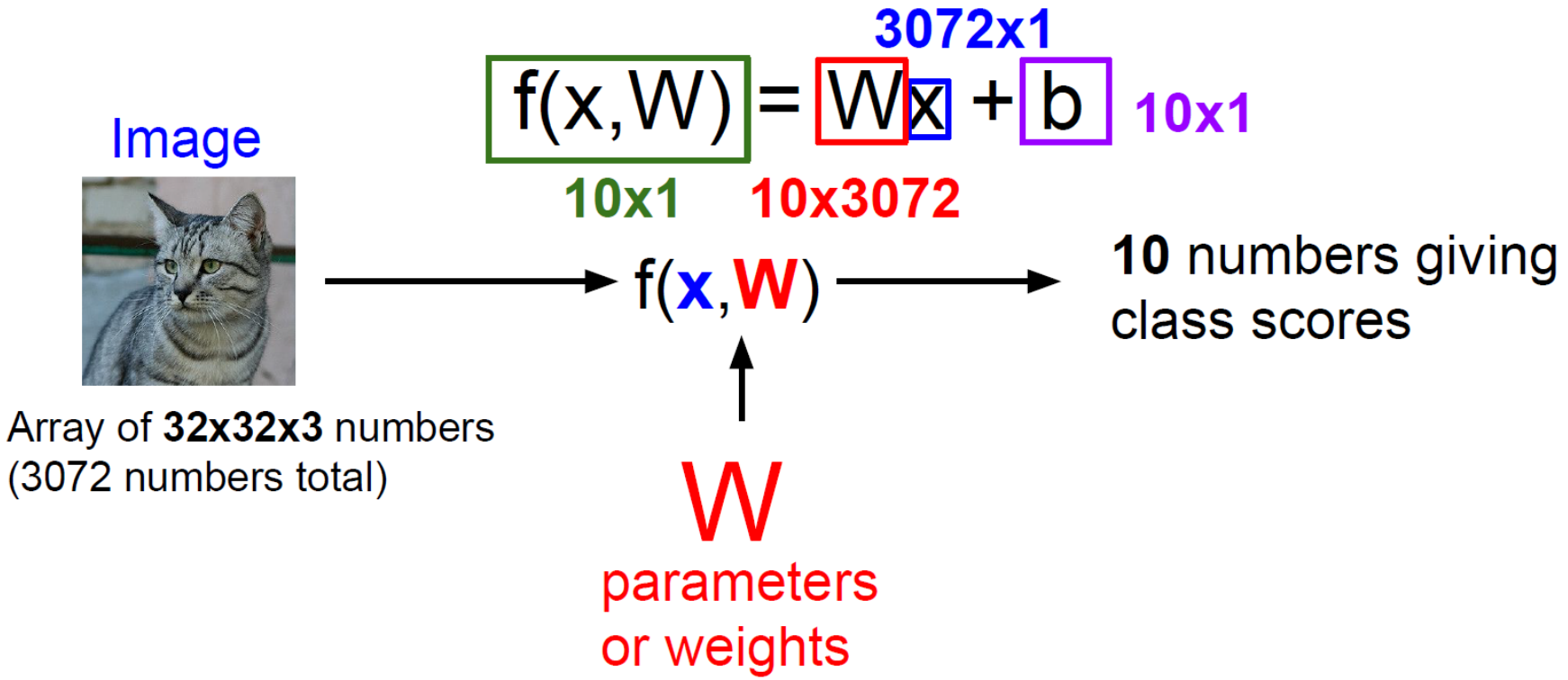


Image Classification with Linear Classifier

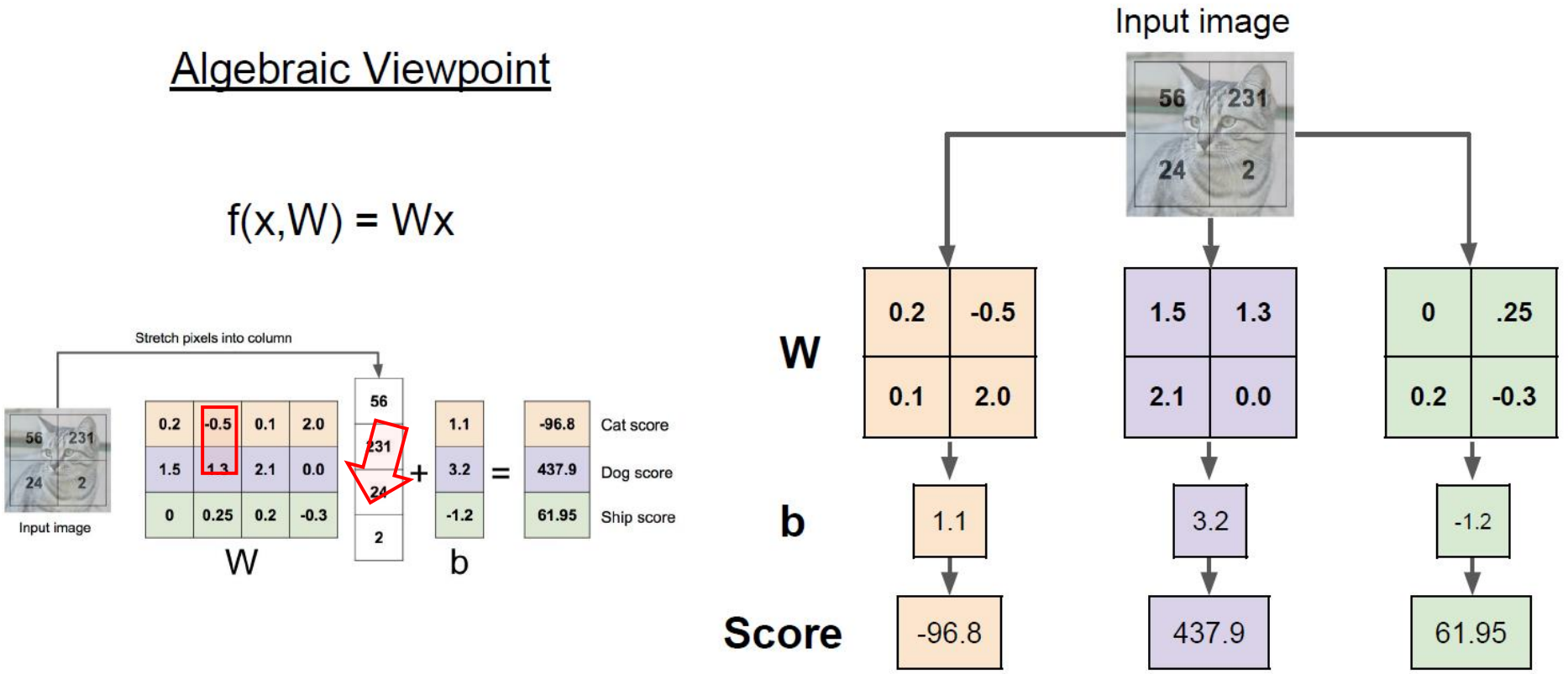


Image Classification with Linear Classifier

e.g., **W** is *m* and *c: f(x,y) = y-mx-c*

Assume m =1 c =-2 (deer classifier):

f(1,1) = 1 –1*1 + 2= 2 > 0 -> above the line

f(-5,0) = 0 -1*5 +2 = -3 < 0 -> below the line

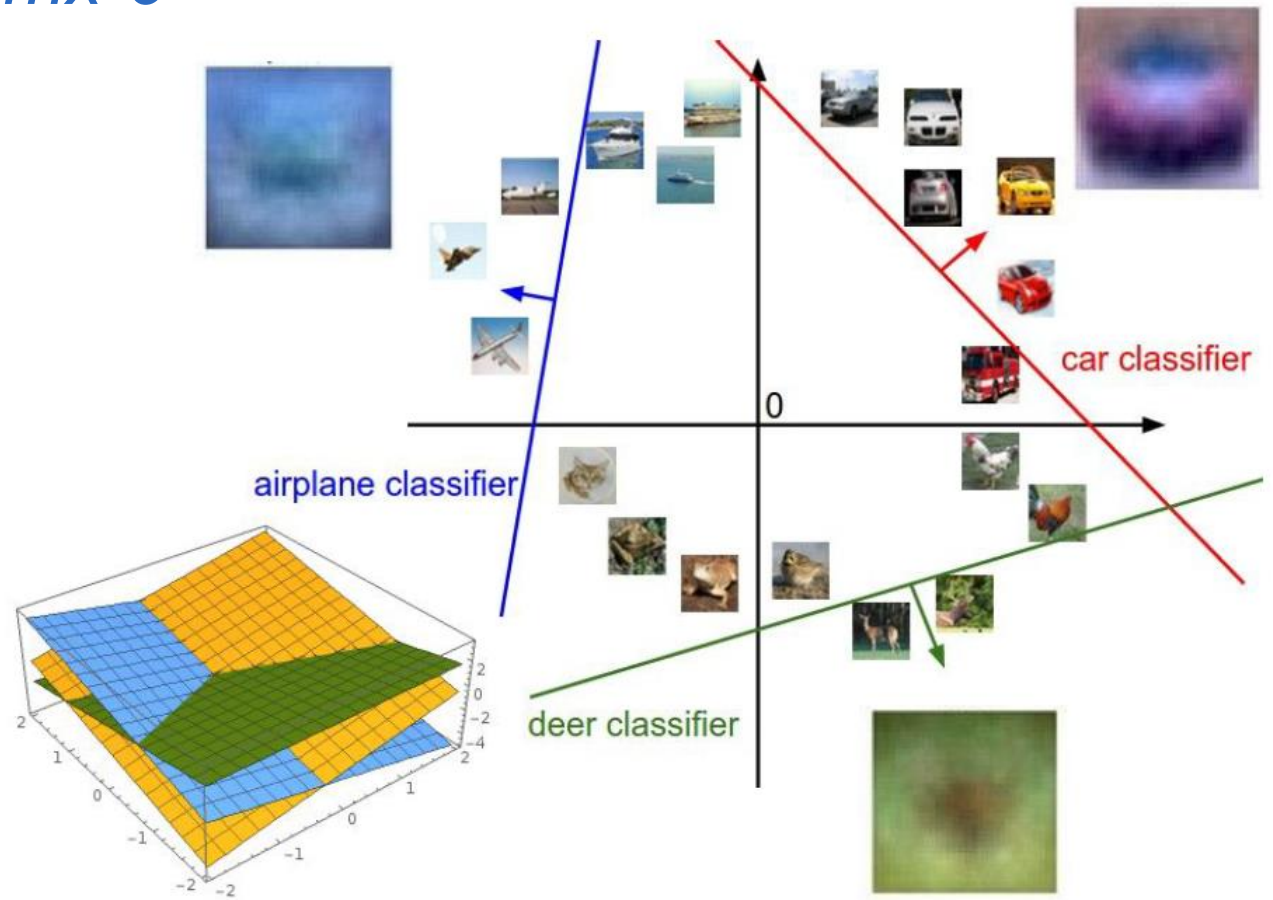
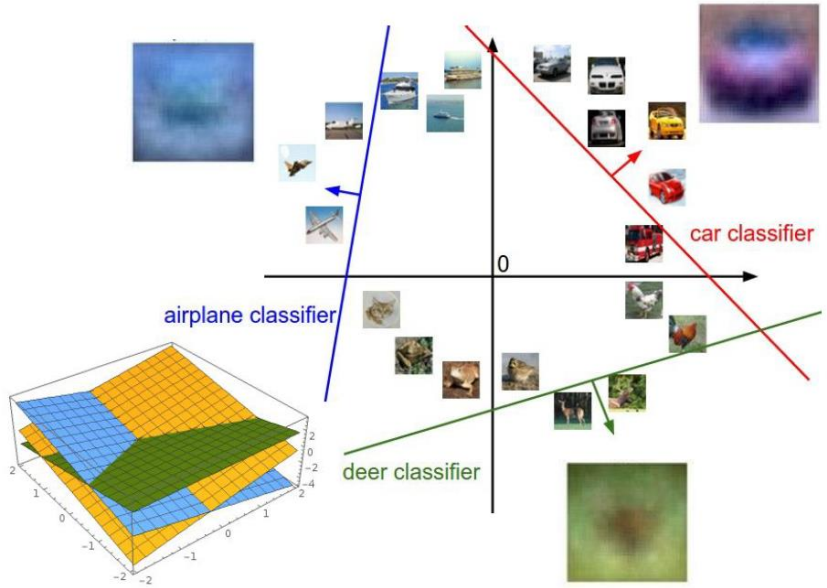


Image Classification with Linear Classifier

- m1, m2, m3, c1, c2, c3 for 3 lines

$$\begin{matrix} -m1 & 1 \\ [-m2 & 1 \\ -m3 & 1 \end{matrix} \begin{matrix} x \\ ][\,y\,] \\ \end{matrix} \begin{matrix} -c1 \\ +[-c2] \\ -c3 \end{matrix} = \begin{matrix} z_1 \\ [z_2] \\ z_3 \end{matrix}$$



- z01 =y-m1*x-c1
- z02 =y-m2*x-c2
- z03 =y-m3*x-c3

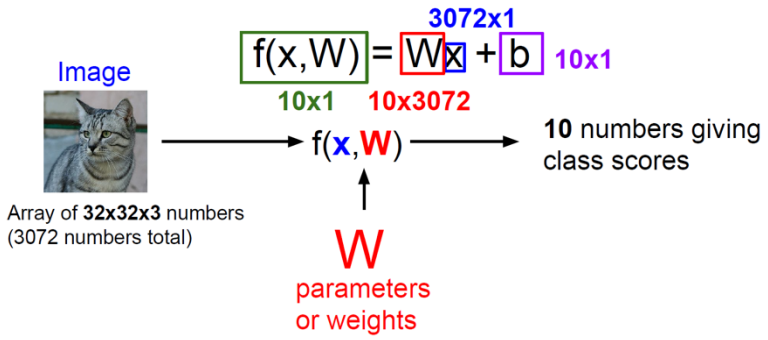


W **bias**

$$\begin{matrix} -m1 & 1 \\ [-m2 & 1 \\ -m3 & 1 \end{matrix} \begin{matrix} x \\ ][\,y\,] \\ \end{matrix} \begin{matrix} -c1 \\ +[-c2] \\ -c3 \end{matrix} = \begin{matrix} z_{01} \\ [z_{02}] \\ z_{03} \end{matrix}$$

input **output**



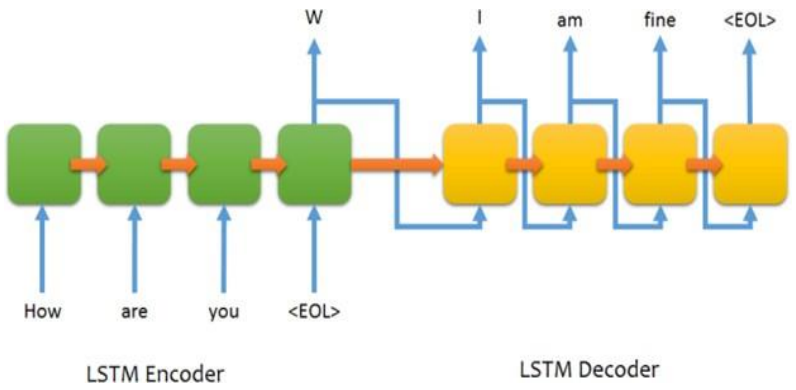


Textual Data

- neural network models for labelled data:

$$\{x_i, y_i\}_{i=1}^{N} \rightarrow P(y|x)$$

where each sample is an **input** x_i and **an output** y_i.



- Each training example consists of **a sequence of inputs**
  - $(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(\tau i)})$ and **a sequence of outputs** $(\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(\tau i)})$,
  - (*"How", "are", "you", –, –, –*), (*–, –, –, "I", "am", "fine"*).

- Training data can be from dialogue or machinetranslation



Textual Data: One Hot Vectors

- a binary vector used to represent categorical /text data
- Each category is represented by a unique index in the vector.
- without any implicit relationships between categories.

Island		Biscoe	Dream	Torgensen
Biscoe	→	1	0	0
Torgensen		0	0	1
Dream		0	1	0



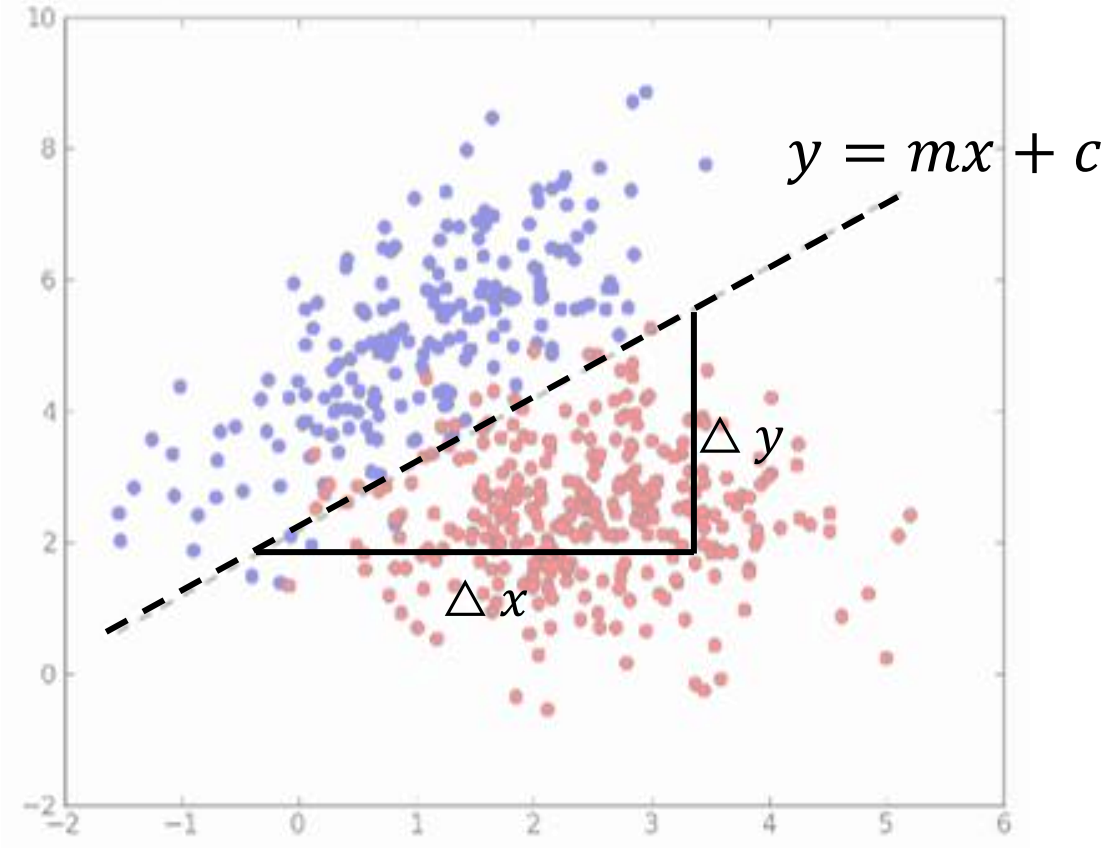
Agenda

- Linear classifier
- Transfer Learning
- Local Large Language Model



Summary so far

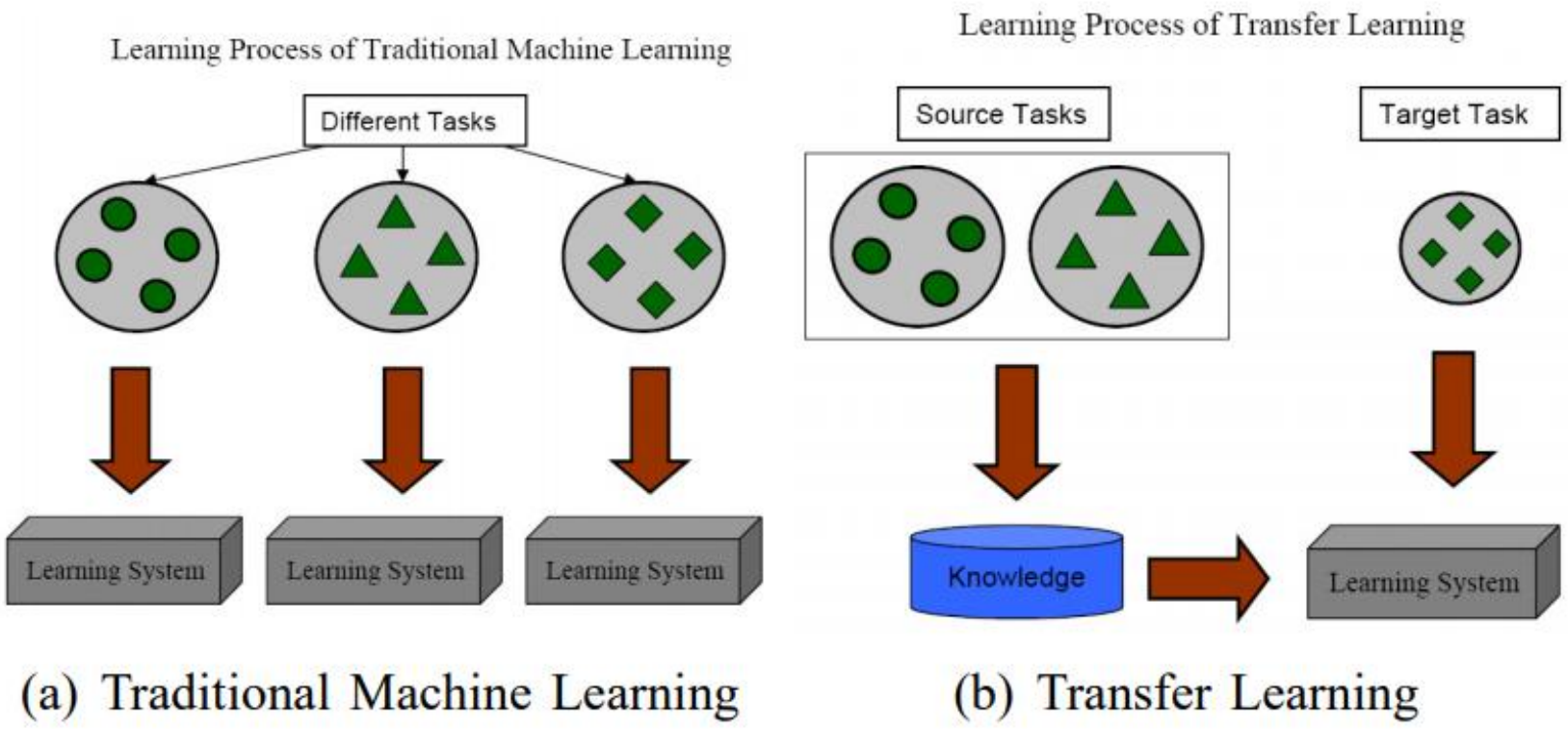
- Data is represented as a point
- Classification is to learn the equation
- What if we don't know how to represent the data?





Transfer Learning

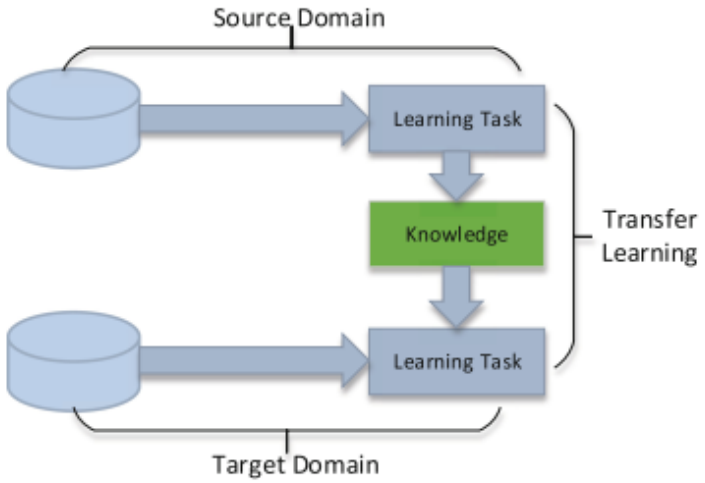
- aims to leverage the learned knowledge from a resource-rich domain/task to help learning a task with not sufficient training data.



Transfer Learning

- The resource-rich domain is known as the **source** and the low-resource task is known as the **target**.

- Transfer learning works the best if the model features learned from the source task are **general** (i.e., domain-independent)
  - e.g., trained from ImageNet





Transfer Learning

Image Processing: Learning flower classification system

- How much data can we have?

Text Sentiment Analysis: Learning a sentiment on camera review

- Need to label many camera reviews from scratch?
- Can we use online datasets?

What are source and target tasks/domain in the above examples?



(Deep) Transfer Learning:

aims to help improve the learning of the target predictive function $f_T(.)$ in D_t using the knowledge in D_s and $\mathcal{T}_s$, where $D_s \neq D_t$ and/or $\mathcal{T}_s \neq \mathcal{T}_t$.

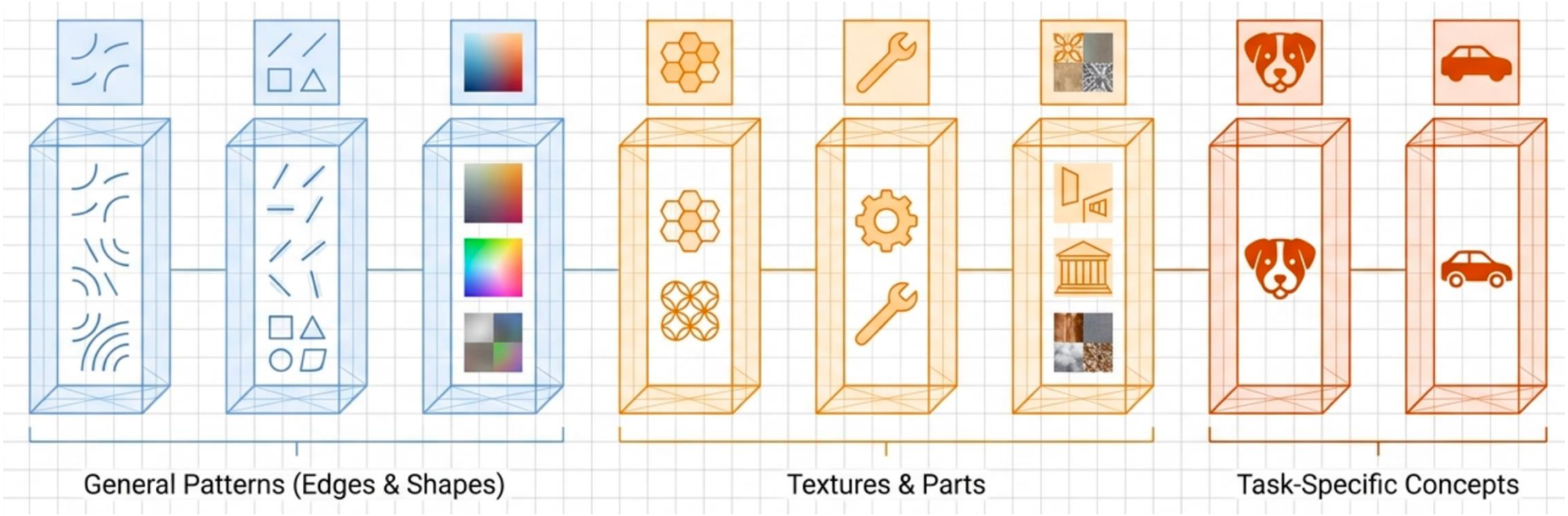
- $\mathcal{D}_s$ and $\mathcal{D}_t$ denote the source and target domains
- $\mathcal{T}_s$ and $\mathcal{T}_t$ be the source and target tasks

- (Deep learning) Models can be pre-trained on one domain



The General-to-Specific Spectrum of Deep Learning

- Deep networks learn hierarchical features. Early layers act as universal feature detectors. Only the deepest layers specialize in the original training task, making early layers highly transferable across domains



Major types of Deep Transfer Learning

- **Representation-based:** use a pre-trained model to encode data into a representation

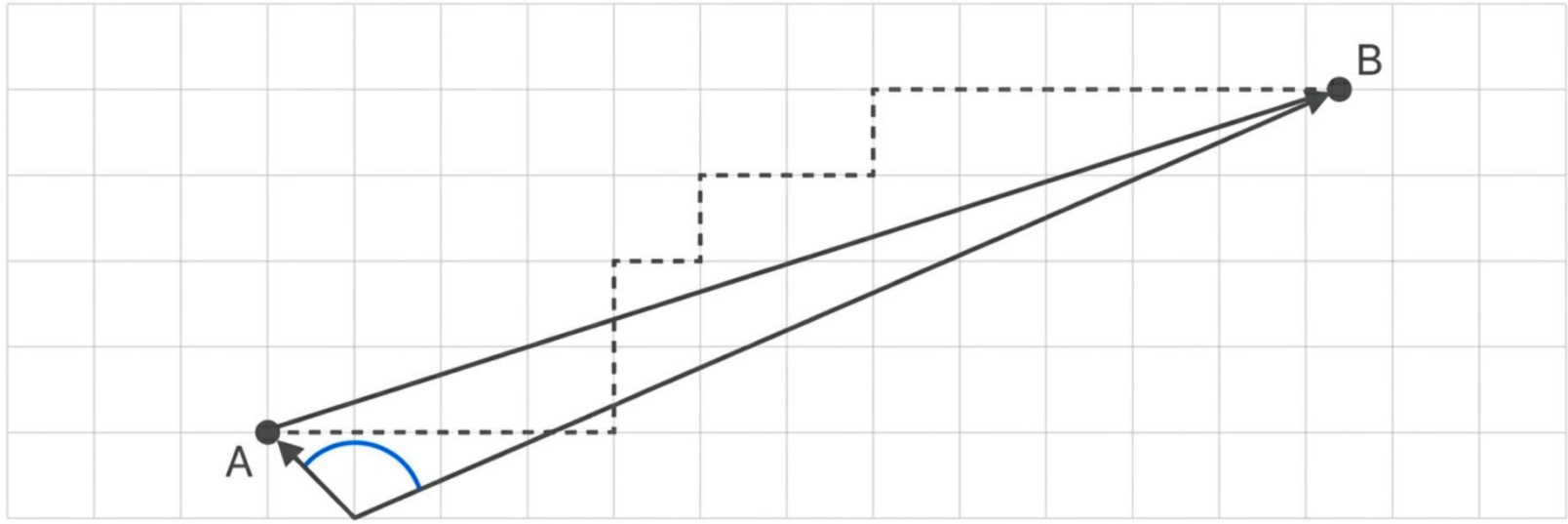
- **Model Sharing-based:** Reuse part of a model that is pre-trained in the source domain

- **Instance-based:** Reuse the instances (samples) in source domain by assigning appropriate weight to them



Recall: First, you need a space

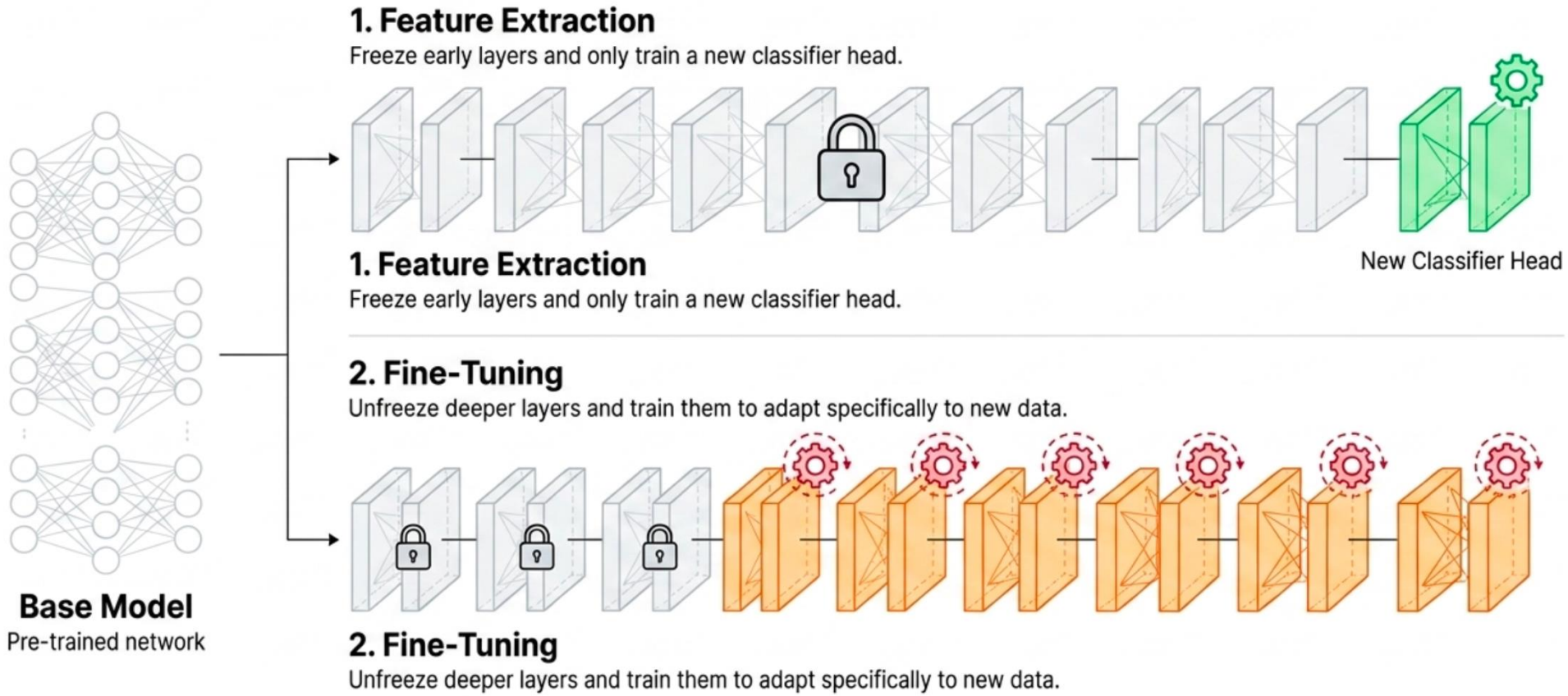
- An algorithm depends on how “closeness” is defined
- Changing the mathematical rules for measuring space fundamentally alters which neighbors are considered closest





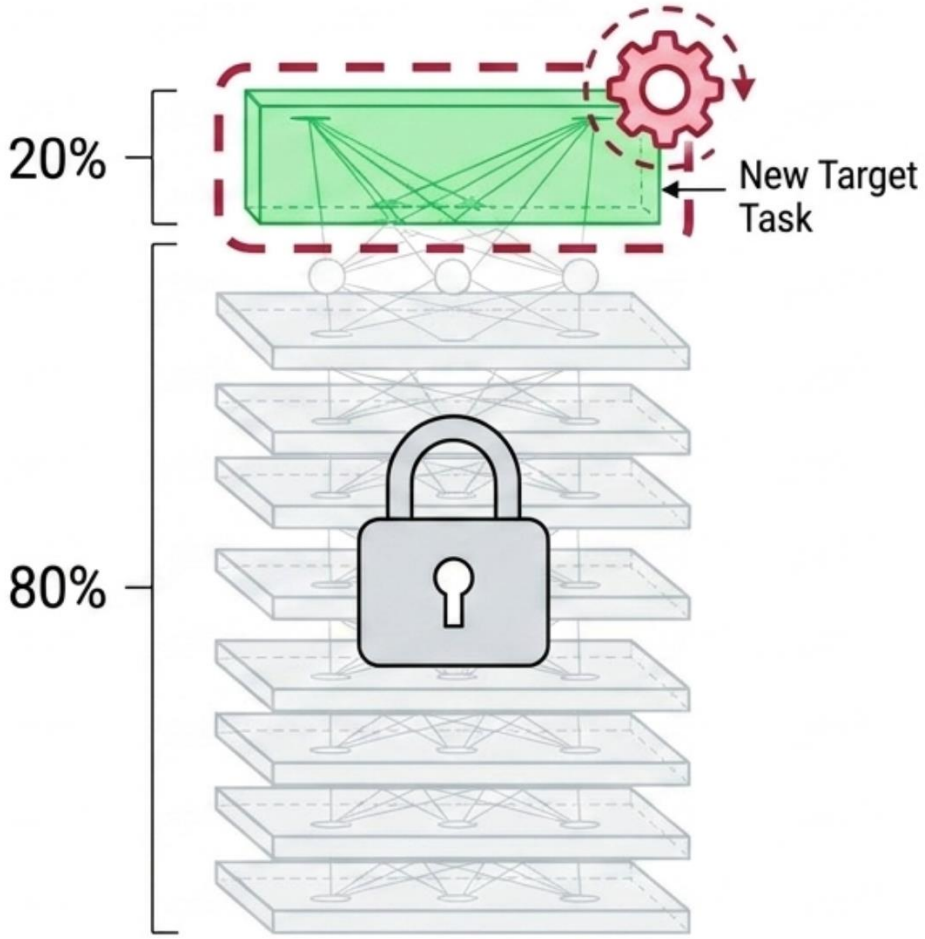
Representation & Model-based

- You can leverage a pre-trained model in two ways



Representation-based Deep Transfer Learning

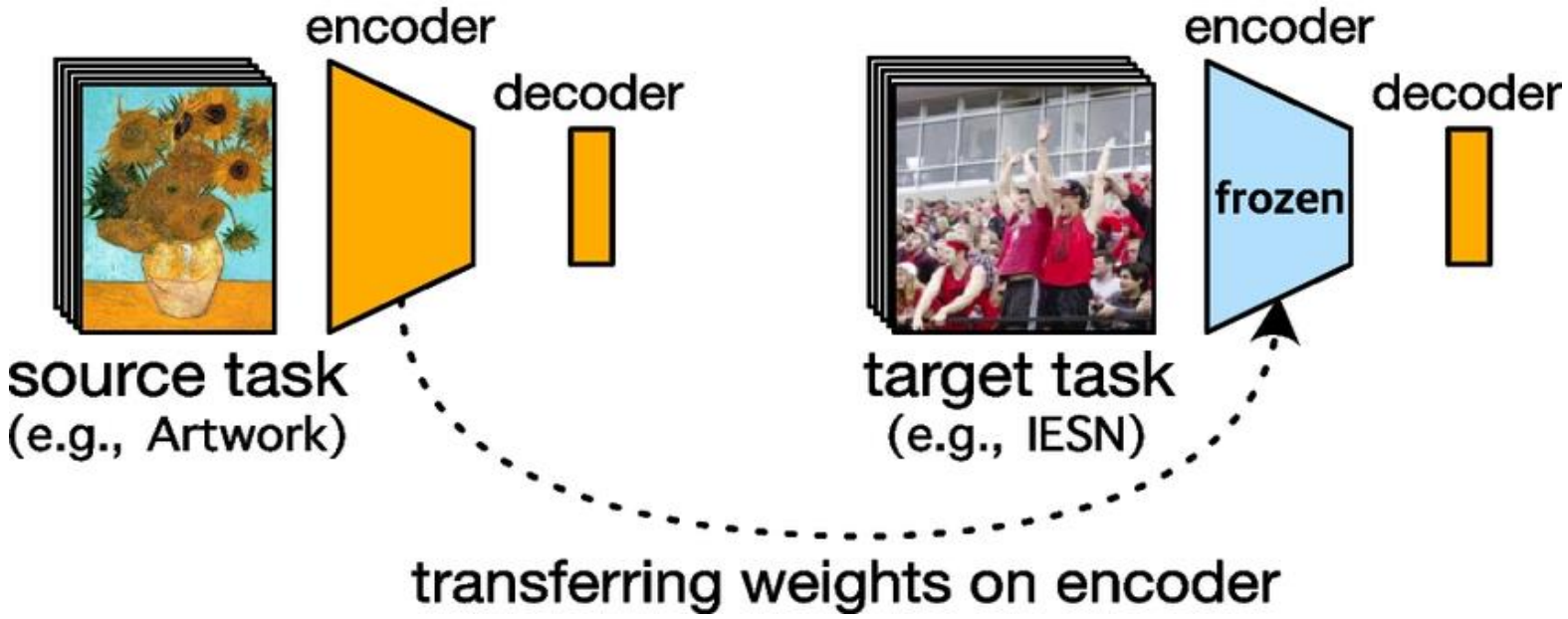
- The pre-trained layers act as a fixed feature map. Optimization is restricted entirely to the newly initialized classifier head.
- Data Requirement: Optimal for very small target datasets (< 1,000 samples).
- Compute: Low memory footprint; gradients are computed for the classifier head only.



Representation-based Deep Transfer Learning

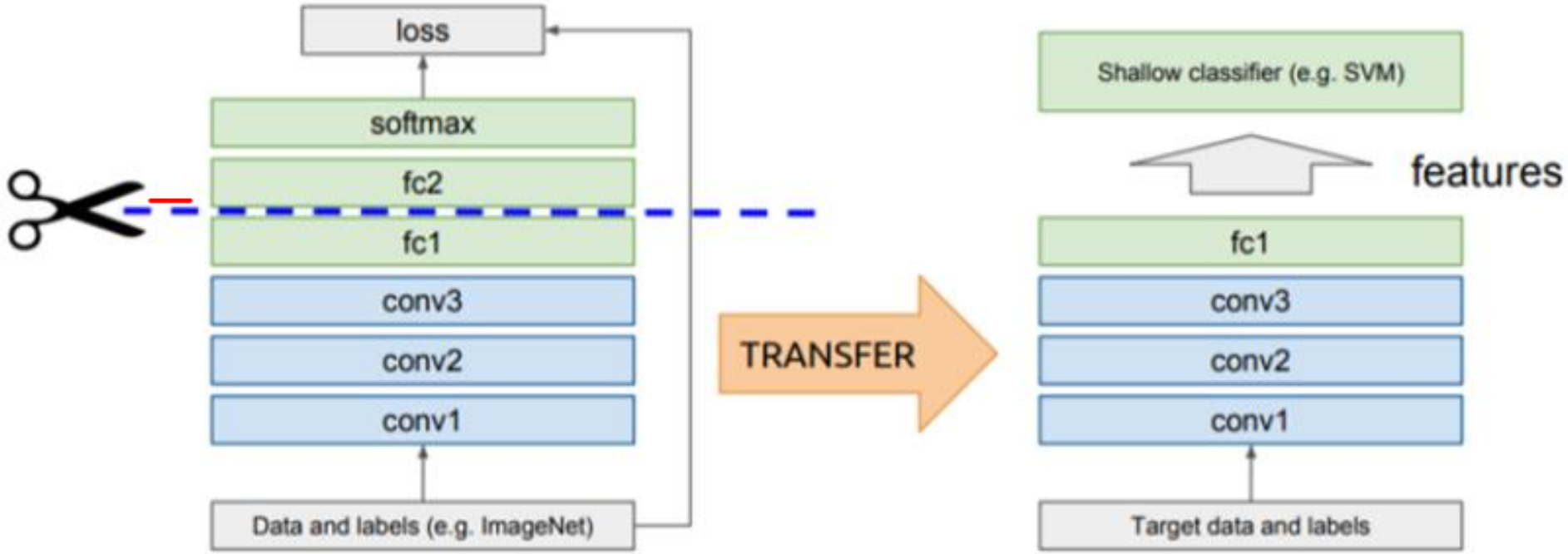
Pre-trained (by others)

Transferred



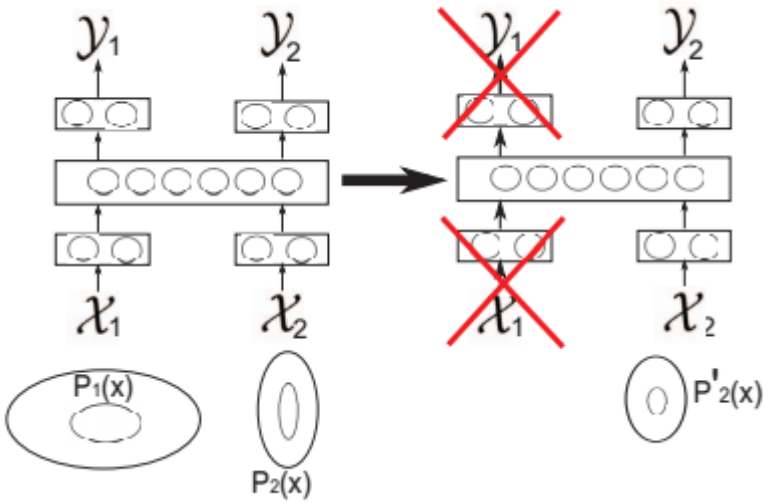
Representation-based Deep Transfer Learning

- Use outputs of one or mare layers to train on another model



Representation-based Deep Transfer Learning

- It is also possible that the entire model is shared and trained on source and target domains **jointly**.
- $\mathcal{X}_1$ and $\mathcal{Y}_1$: feature and label spaces for the learning task in the source domain
- $\mathcal{X}_2$ and $\mathcal{Y}_2$: feature and label spaces for the learning task in the target domain.
- At the runtime, only the target domain is concerned.



Representation-based Deep Transfer Learning

The layers of a trained model can be treated as feature extractor to extract high-level features from a new domain

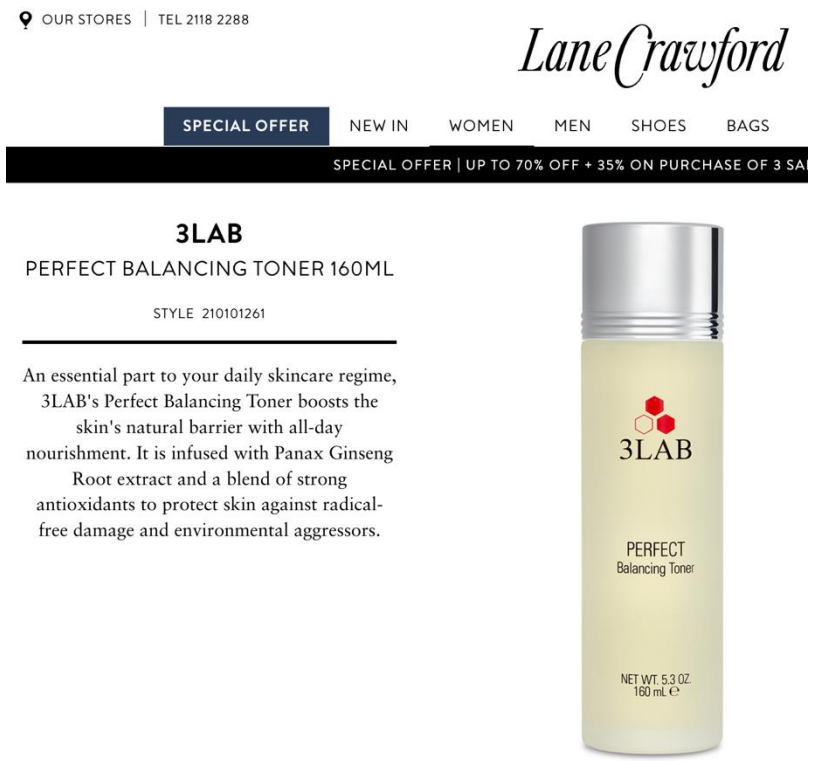
Approach:

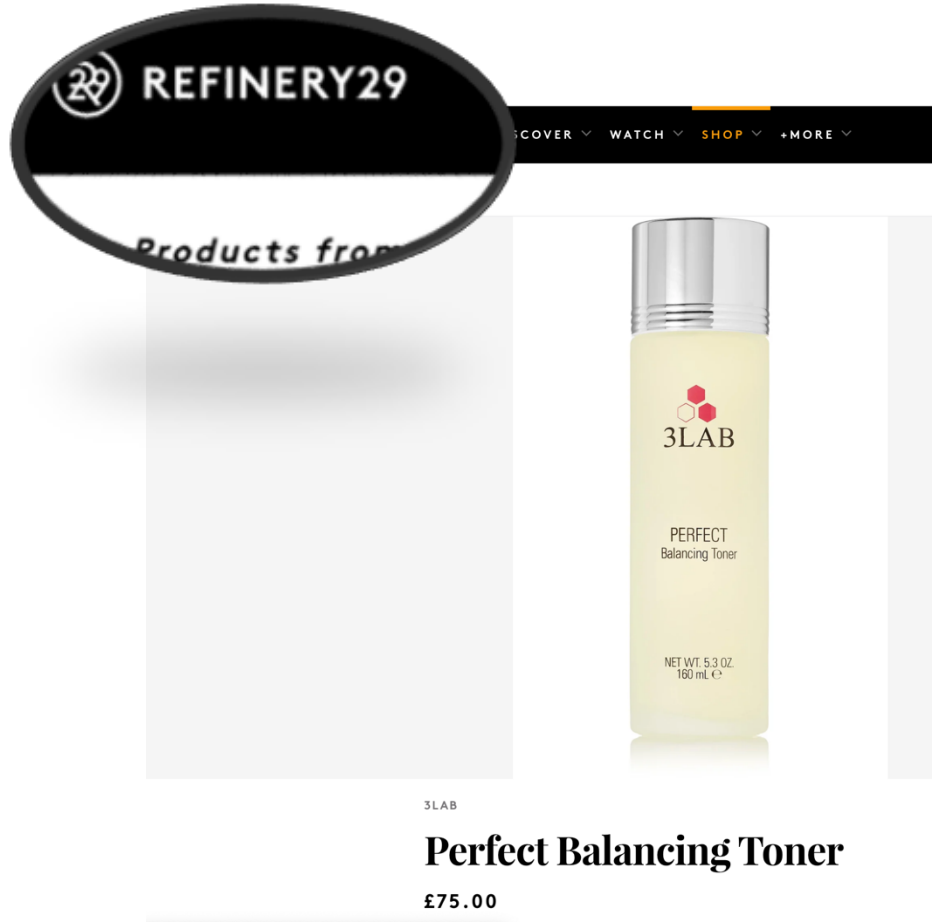
1. The target domain are encoded using the pre-trained encoder using source domain
2. Instances in the new feature space can be used as the training set of a new neural network



Representation-based Deep Transfer Learning

- Using common items to learn a representation for all items





Textual

	Model	IR (DPR) BEIR	IR (DPR) $MLDR_{OOD}$	IR (DPR) $MLDR_{ID}$	IR (ColBERT) BEIR	IR (ColBERT) $MLDR_{OOD}$	NLU GLUE	Code CSN	Code SQA
Base	BERT	38.9	23.9	32.2	49.0	28.1	84.7	41.2	59.5
	RoBERTa	37.7	22.9	32.8	48.7	28.2	86.4	44.3	59.6
	DeBERTaV3	20.2	5.4	13.4	47.1	21.9	88.1	17.5	18.6
	NomicBERT	41.0	26.7	30.3	49.9	61.3	84.0	41.6	61.4
	GTE-en-MLM	41.4	**34.3**	**44.4**	48.2	69.3	85.6	44.9	71.4
	ModernBERT	**41.6**	27.4	44.0	**51.3**	**80.2**	**88.5**	**56.4**	**73.6**
Large	BERT	38.9	23.3	31.7	49.5	28.5	85.2	41.6	60.8
	RoBERTa	41.4	22.6	36.1	49.8	28.8	88.9	47.3	68.1
	DeBERTaV3	25.6	7.1	19.2	46.7	23.0	**91.4**	21.2	19.7
	GTE-en-MLM	42.5	**36.4**	**48.9**	50.7	71.3	87.6	40.5	66.9
	ModernBERT	**44.0**	34.3	48.6	**52.4**	**80.4**	90.4	**59.5**	**83.9**



Ref: **Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference**

Textual

- ModernBERT supports up to 8,192 tokens, a significant increase from BERT's 512, enabling it to handle longer documents effectively.
- It incorporates advanced techniques like Rotary Positional Embeddings and GeGLU layers, enhancing performance and flexibility.
- The model is optimized for speed and memory efficiency, making it practical for real-world applications on common GPUs.



https://medium.com/@simeon.emanuilov/modernbert-a-modernized-bert-for-nlp-tasks-unfoldai-0ca2c66bc669

```python
def encode_text(text, model_name='answerdotai/ModernBERT-base', return_cls=True, use_fp16=False,
device='cpu'):
    """
    Encode text using ModernBERT model.

    Args:
        text (str): Input text to encode.
        model_name (str): Name of the ModernBERT model (default: 'answerdotai/ModernBERT-base').
        return_cls (bool): If True, return CLS token embedding; else, return all token embeddings.
        use_fp16 (bool): If True, use mixed precision (float16) for faster inference.
        device (str): Device to run the model on ('cpu' or 'cuda').

    Returns:
        numpy.ndarray: CLS token embedding or all token embeddings.
    """
    # Load pre-trained ModernBERT tokenizer and model
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = ModernBertModel.from_pretrained(model_name)

    # Move model to specified device
    model = model.to(device)

    # Tokenize input text
    inputs = tokenizer(text, return_tensors='pt', padding=True, truncation=True, max_length=8192)

    # Move inputs to device
    inputs = {k: v.to(device) for k, v in inputs.items()}

    # Get model outputs (embeddings)
    with torch.no_grad():
        outputs = model(**inputs)

    # Extract embeddings (last hidden state)
    embeddings = outputs.last_hidden_state
```



Text and audio

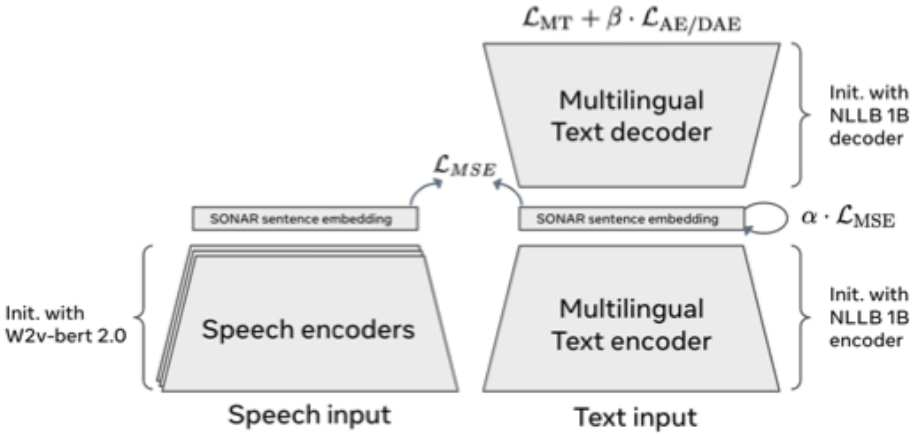


Figure 1: SONAR architecture.

Compute speech sentence embeddings with SONAR

```
from sonar.inference_pipelines.speech import SpeechToEmbeddingModelPipeline
s2vec_model = SpeechToEmbeddingModelPipeline(encoder="sonar_speech_encoder_eng")

s2vec_model.predict(["./tests/integration_tests/data/audio_files/audio_1.wav",
                     "./tests/integration_tests/data/audio_files/audio_2.wav"]).shape
# torch.Size([2, 1024])
import torchaudio
inp, sr = torchaudio.load("./tests/integration_tests/data/audio_files/audio_1.wav")
assert sr == 16000, "Sample rate should be 16kHz"

s2vec_model.predict([inp]).shape
# torch.Size([1, 1024])
```

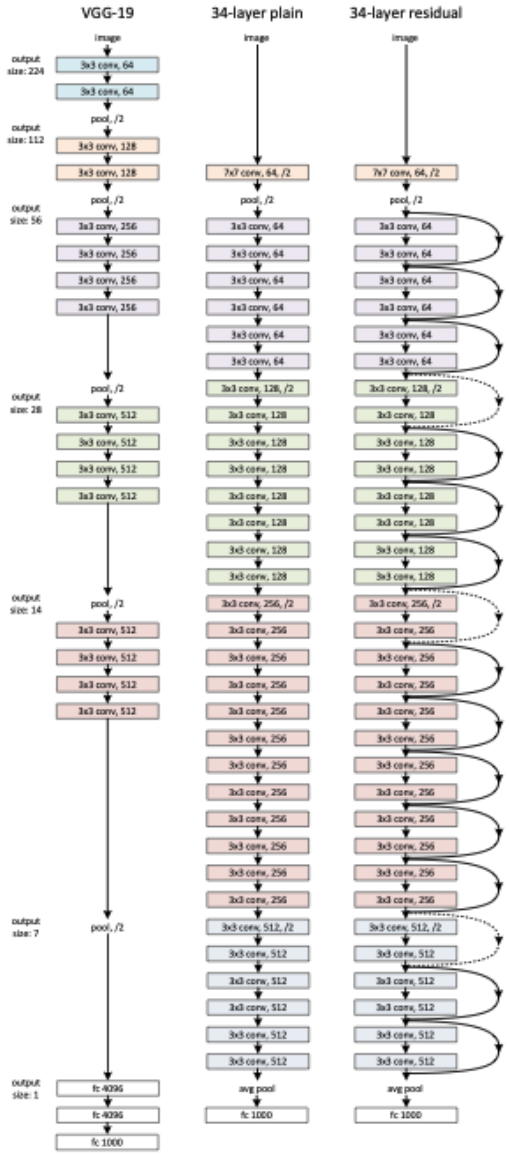
ISO	Language	Train	Ours	Whisper
arb	MS Arabic	822	30.9	26.9
ben	Bengali	335	21.3	14.1
cat	Catalan	1738	37.7	36.9
cmn	Mandarin Chinese	9320	18.6	20.8
ces	Czech	181	32.0	30.3
cym	Welsh	99	14.5	13.4
dan	Danish	115	34.9	36.0
deu	German	3329	36.2	38.8
est	Estonian	131	26.1	21.2
fin	Finish	184	24.9	25.2
fra	French	2057	33.7	34.9
hin	Hindi	150	22.6	24.2
ind	Indonesian	269	28.7	31.9
ita	Italian	588	29.3	27.5
jpn	Japanese	17319	20.2	20.8
kan	Kannada	114	21.4	13.1
kor	Korean	316	17.1	24.2
mlt	Maltese	106	24.4	16.2
nld	Dutch	1723	29.3	28.4
pes	Western Persian	386	24.4	20.9
por	Portuguese	269	38.3	41.4
pol	Polish	304	21.1	25.8
ron	Romanian	135	34.7	34.1
rus	Russian	259	28.4	31.1
slk	Slovak	102	32.3	29.3
spa	Spanish	1511	28.0	27.2
swh	Swahili	361	23.5	7.6
tam	Tamil	245	16.2	10.0
tel	Telugu	84	18.0	14.7
tgl	Tagalog	108	14.6	26.8
tha	Thai	195	16.9	17.8
tur	Turkish	174	22.7	29.9
ukr	Ukrainian	105	30.7	32.5
urd	Urdu	185	19.7	18.1
uzn	Uzbek	115	20.0	6.6
vie	Vietnamese	194	19.1	21.9
Total/avg		43628	25.3	24.5

Table 10: spBLEU evaluation of S2T into English on FLEURS test set. Our models were trained on ASR transcriptions only, compared to the Whisper large v2.



Ref: **SONAR: Sentence-Level Multimodal and Language-Agnostic Representations**
Github: https://github.com/facebookresearch/SONAR

Images



method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43†
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except † reported on the test set).

Ref: Deep Residual Learning for Image Recognition

```python
def load_and_preprocess_image(image_path):
    # Define the transformation pipeline
    transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    # Load and preprocess the image
    image = Image.open(image_path).convert('RGB')
    image = transform(image)
    image = image.unsqueeze(0)  # Add batch dimension
    return image
```

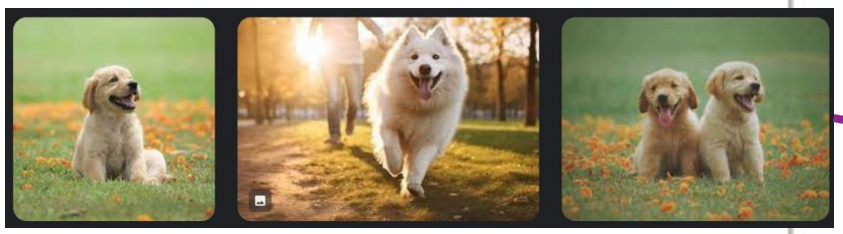
```python
def encode_image(image_path, model, device):
    # Load and preprocess the image
    image = load_and_preprocess_image(image_path)
    image = image.to(device)

    # Set model to evaluation mode
    model.eval()

    # Disable gradient computation for inference
    with torch.no_grad():
        # Forward pass to get the features
        features = model(image)

    # Convert to numpy array and flatten
    features = features.cpu().numpy().flatten()
    return features
```

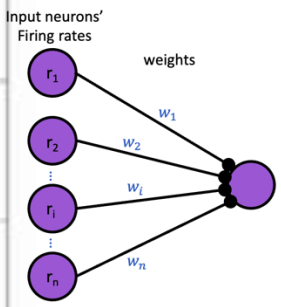


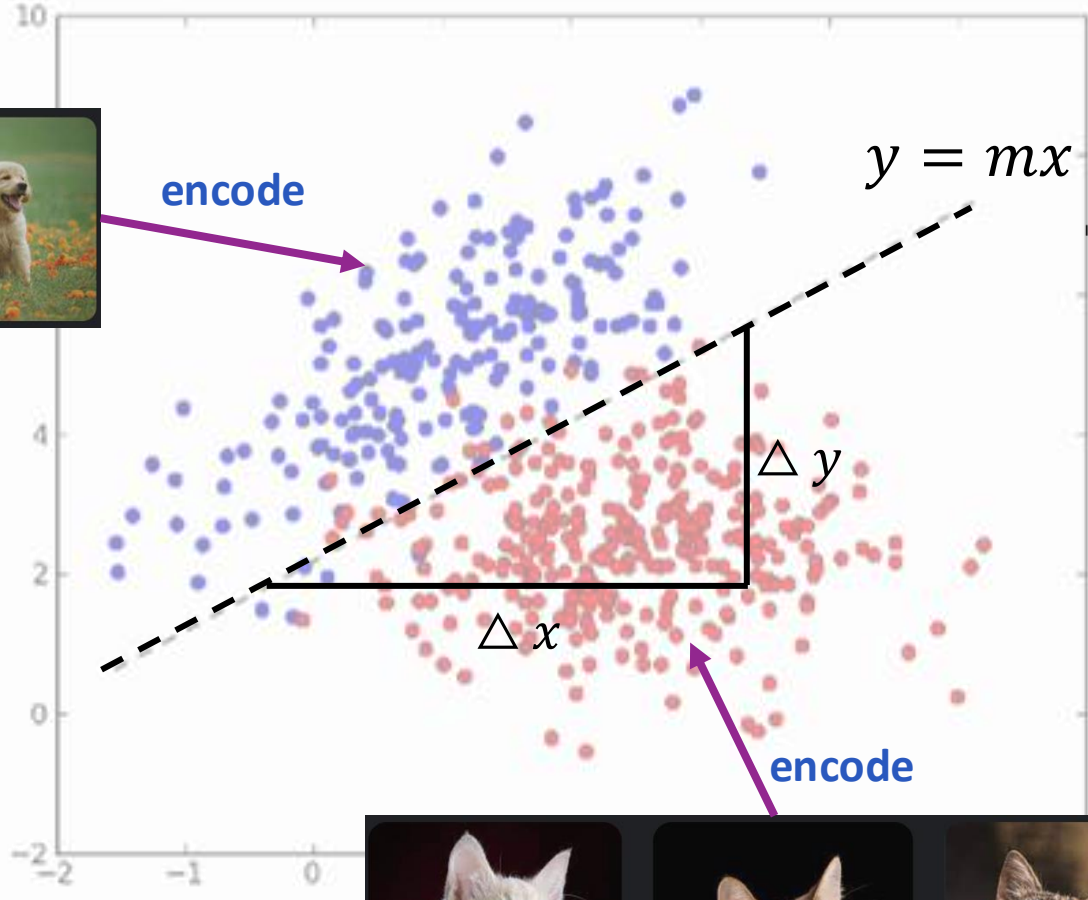


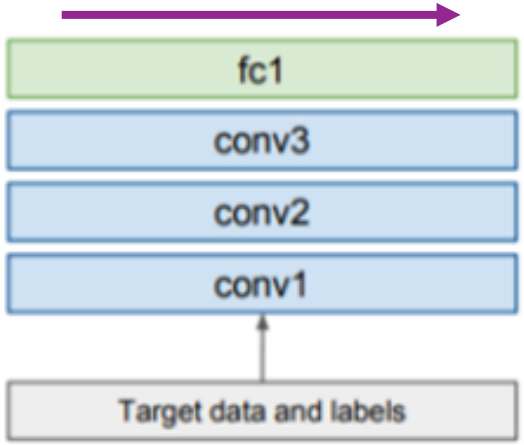
encode

classifier

$y = mx + c$

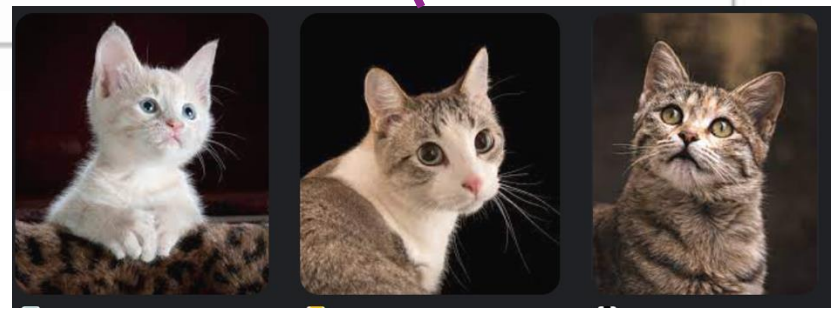






Pre-train model (encoder)

encode



Model Sharing-Based Deep Transfer Learning

The layers of a trained model can be fine-tuned

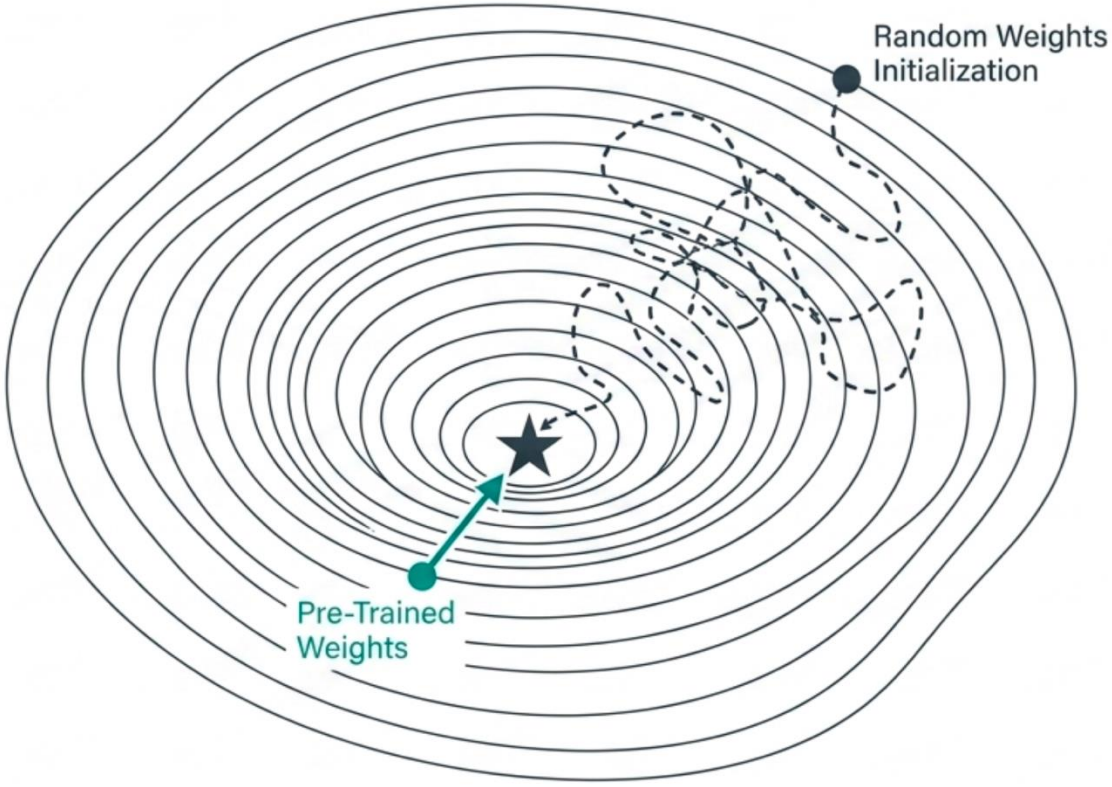
Approach:

1. Find or train a model in source domain with large-scale training dataset
2. Part/all of the pre-trained network is reused in the architecture of the model for target model
3. Weights of the reused part can be updated through training on the target domain (a.k.a. fine-tuning)

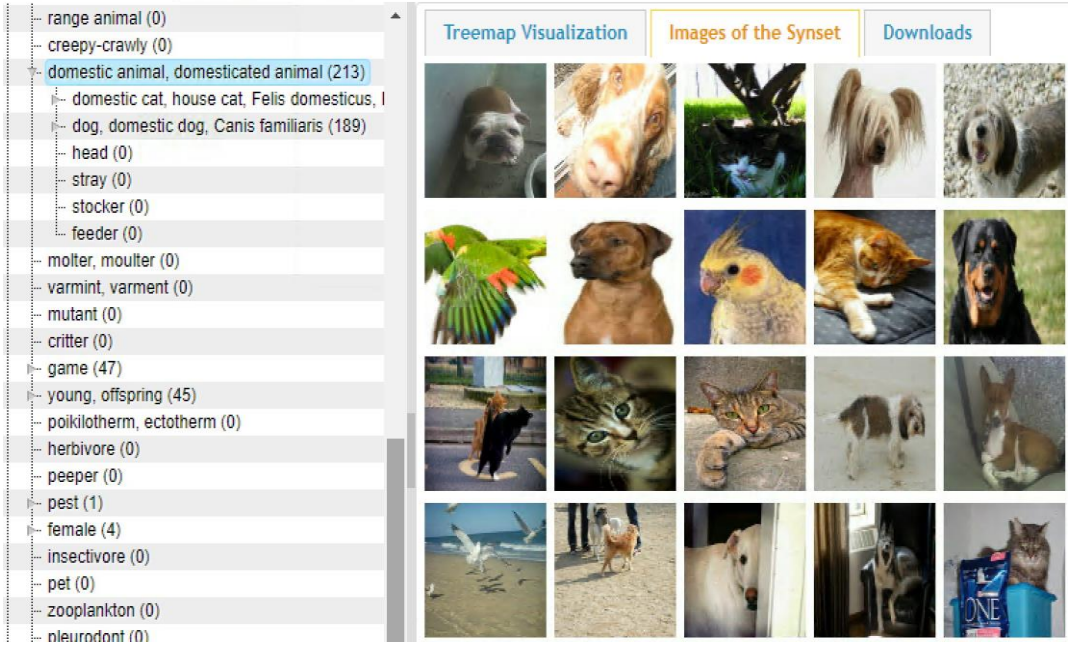


The Paradigm Shift: Standing on the Shoulders of Giants

- Instead of training from scratch, transfer learning takes a model already trained on a massive dataset (like ImageNet for vision, or BERT for text) and uses its learned features as a starting point.
- The Result: 80-90% reduction in required training data and compute costs. You bypass the phase of teaching the network basic reality.



Model Sharing-Based Deep Transfer Learning

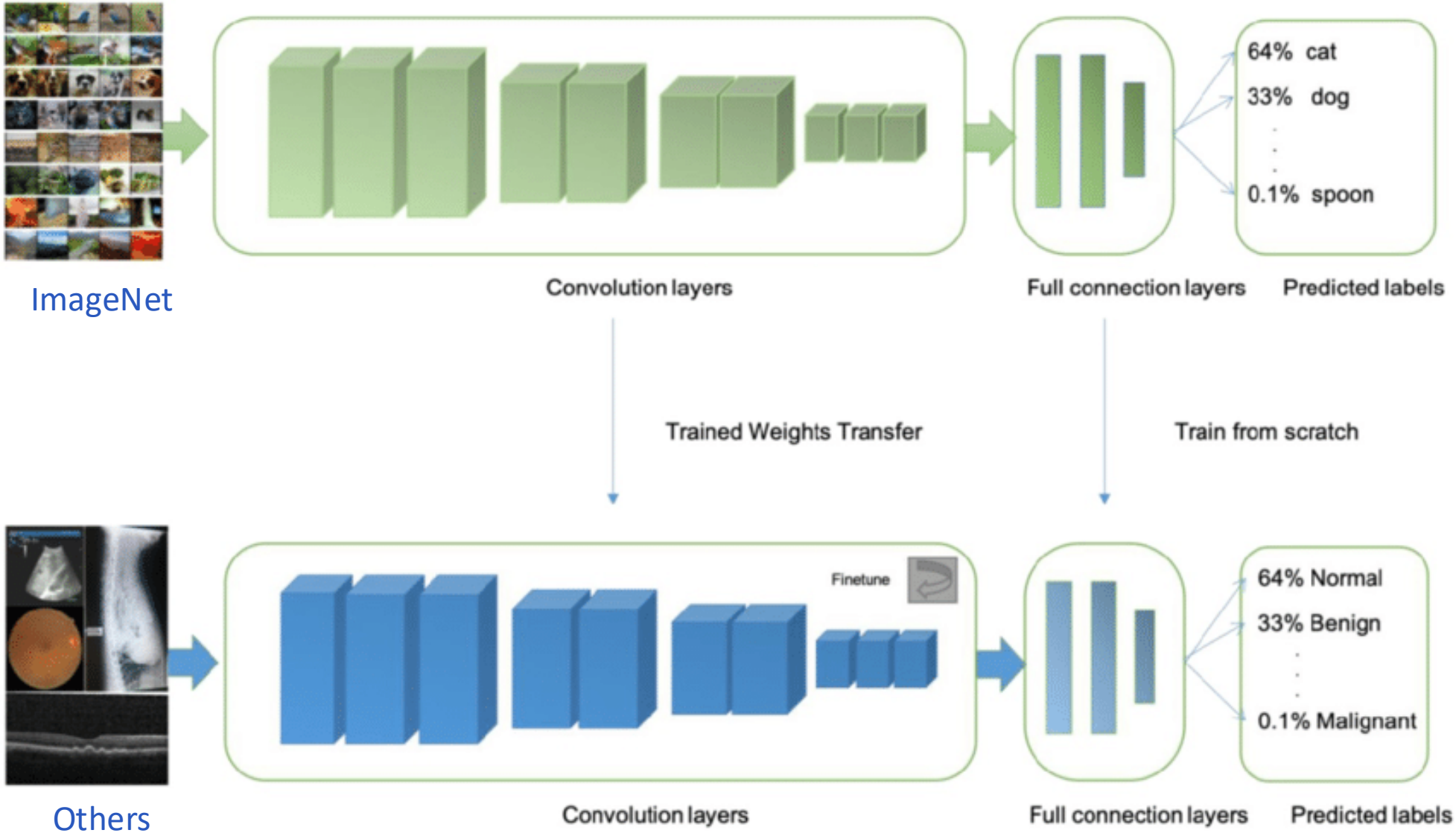


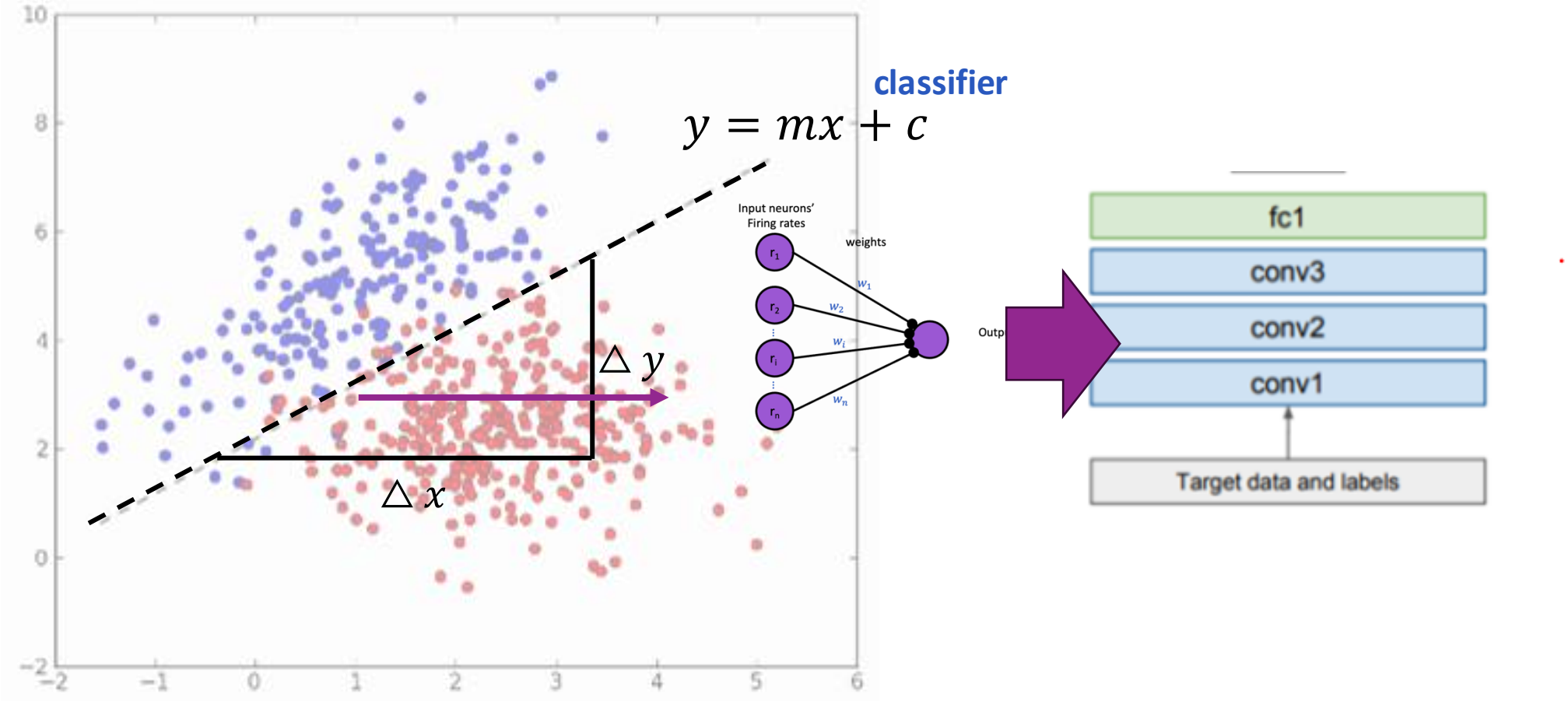
trained from ImageNet

Model



trained from your data

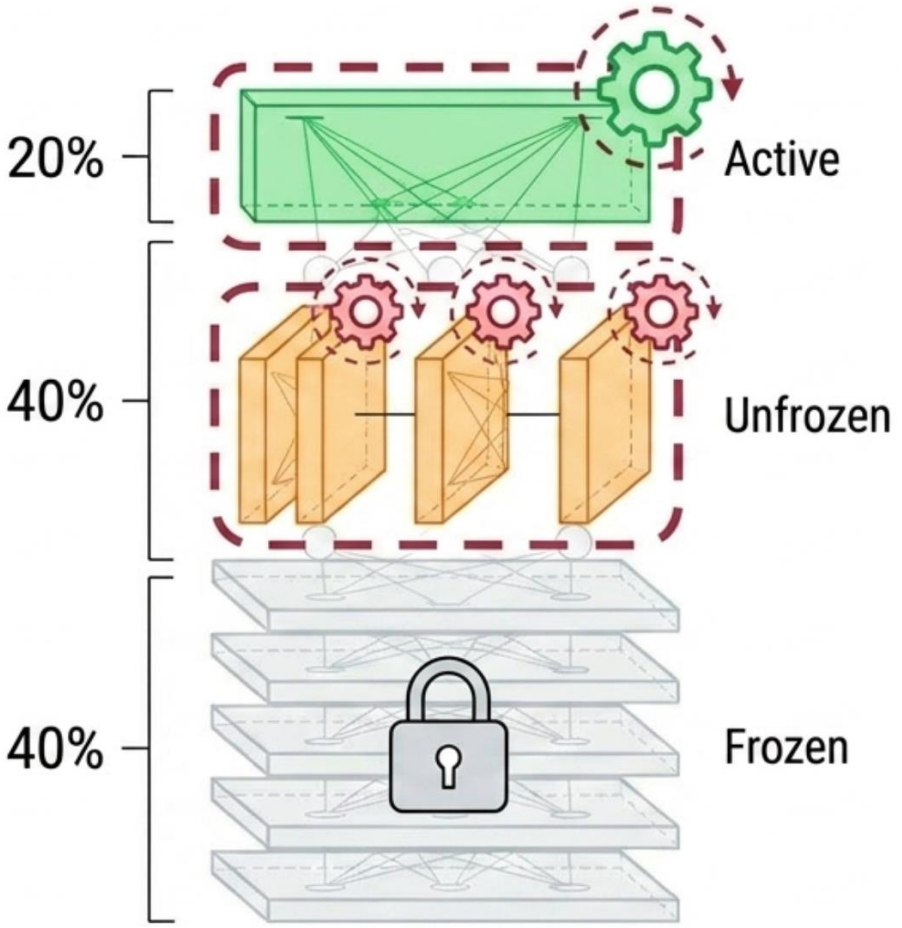

ImageNet
Convolution layers
Full connection layers
64% cat
33% dog
0.1% spoon
Predicted labels
Trained Weights Transfer
Train from scratch
Finetune
Others
Convolution layers
Full connection layers
64% Normal
33% Benign
0.1% Malignant
Predicted labels



Note: all 3 method can be applied together

Partial and Full Fine-Tuning

- Definition: Unfreezing some or all of the deeper pre-trained layers to actively adapt the pre-trained weights to a new domain.
- Data Requirement: Requires a larger dataset (1,000 to 50,000 samples) to prevent overfitting.
- Compute: Medium to high memory footprint; requires gradient computation across the unfrozen backbone.



The Architectural Decision Matrix

Criterion	Feature Extraction	Fine-Tuning	Train From Scratch
Target Data Volume	< 1,000 samples	1,000 to 50,000 samples	> 50,000 samples
Domain Similarity	High semantic overlap	Moderate to low similarity	Very low or unrelated
Overfitting Risk	Minimal	Medium to High	High
GPU Memory Footprint	Low	Medium to High	Maximum



Many pre-train models

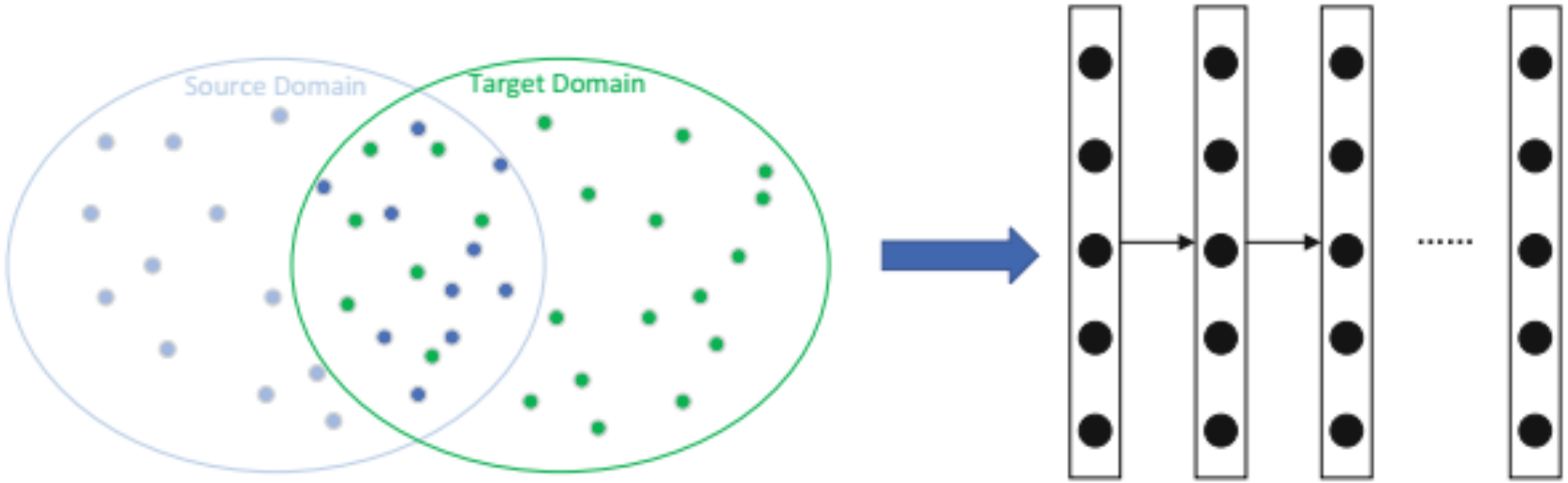
- https://modelzoo.co/
- https://www.tensorflow.org/resources/models-datasets
- https://pytorch.org/hub/
- https://paperswithcode.com/
- Or you can search from google direct



Instance-based Deep Transfer Learning

- Instances shown in light blue in the source domain are not similar to the target domain and therefore are excluded from the training dataset

- Similar instances in the source domain (dark blue) are included in the training dataset with appropriate weights



Instance-based Deep Transfer Learning

- Reweight the samples in the source domain to correct for marginal distribution differences
- Use only the relevant information from the source domain
- Work best when the conditional distribution is the same in both domains.
- **Methods**
  1. finds and applies the weight that matches up the mean of the target and source domains.
  2. train a binary classifier to separate source samples from target samples that the output is the sample weights



Instance-based Deep Transfer Learning

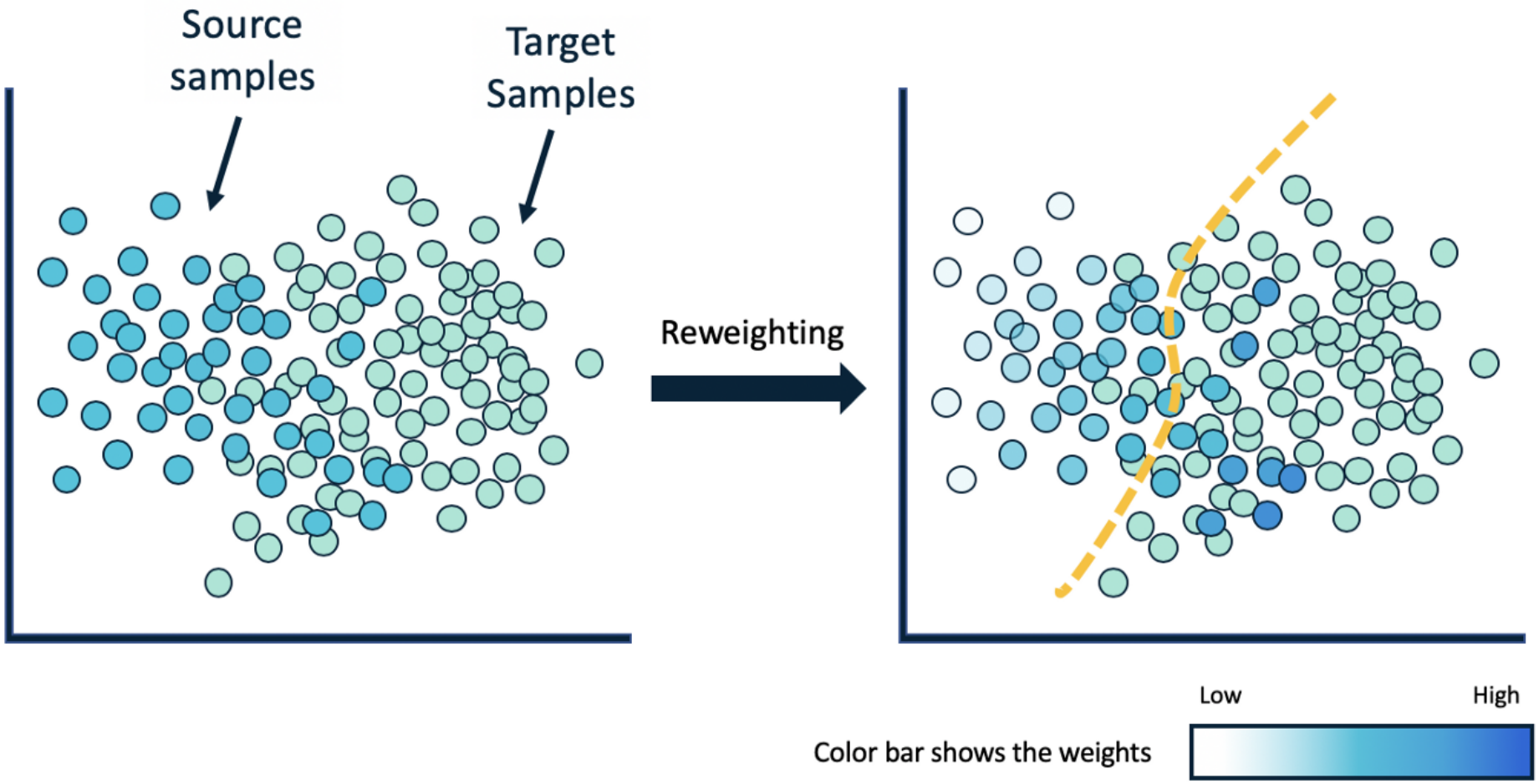
‘Some’ samples in the source domain can be utilized in the target domain with appropriate weights.

Approach:

1. Filtering out the samples that are not similar to the target.
2. Re-weighting instances in source domain to obtain a distribution similar to target domain.
3. Training a deep neural network on the collection of re-weighted instances as well as labeled instances (if any) in the target domain.

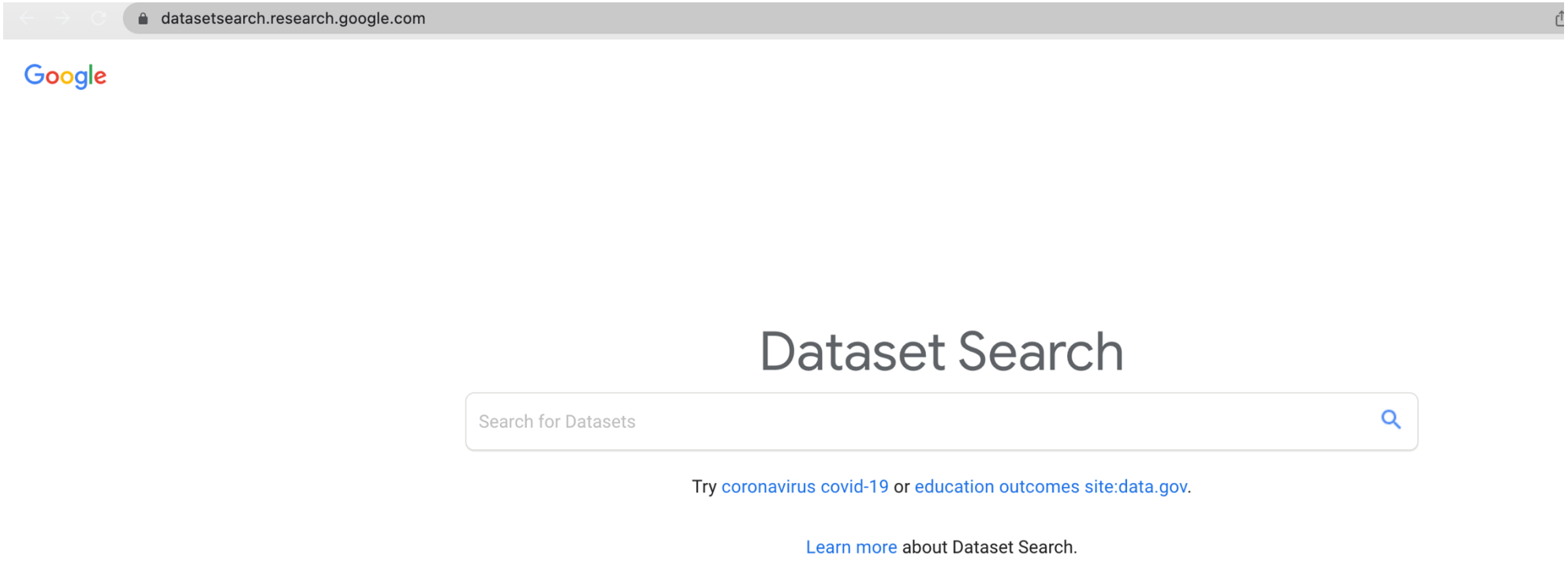


Instance-based Deep Transfer Learning



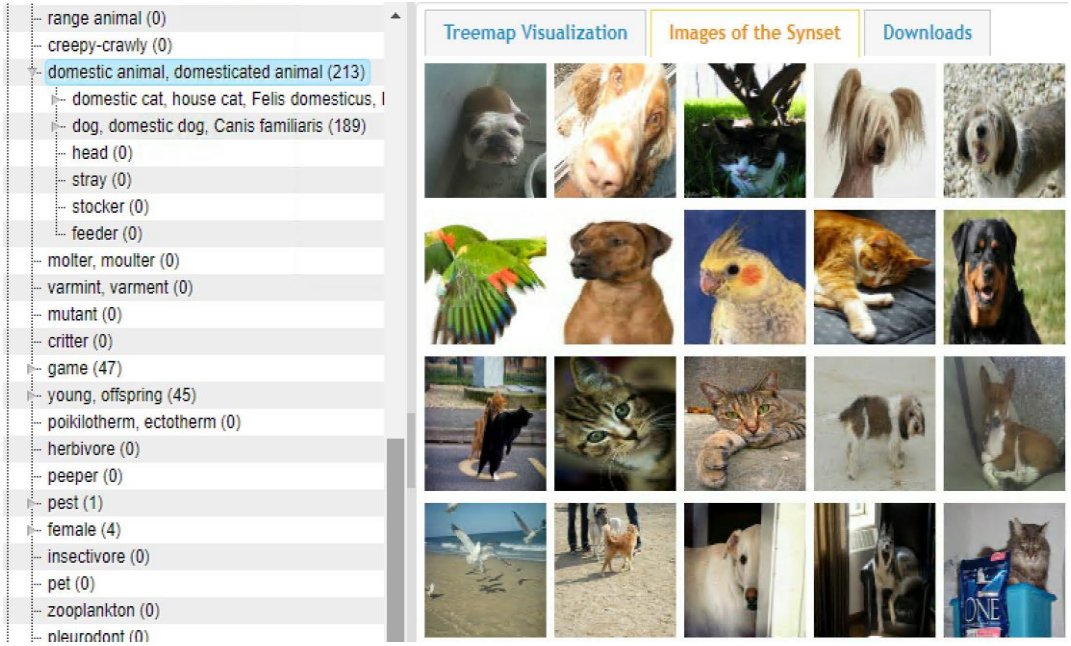
Many open datasets

- https://datasetsearch.research.google.com/





Instance-based Deep Transfer Learning

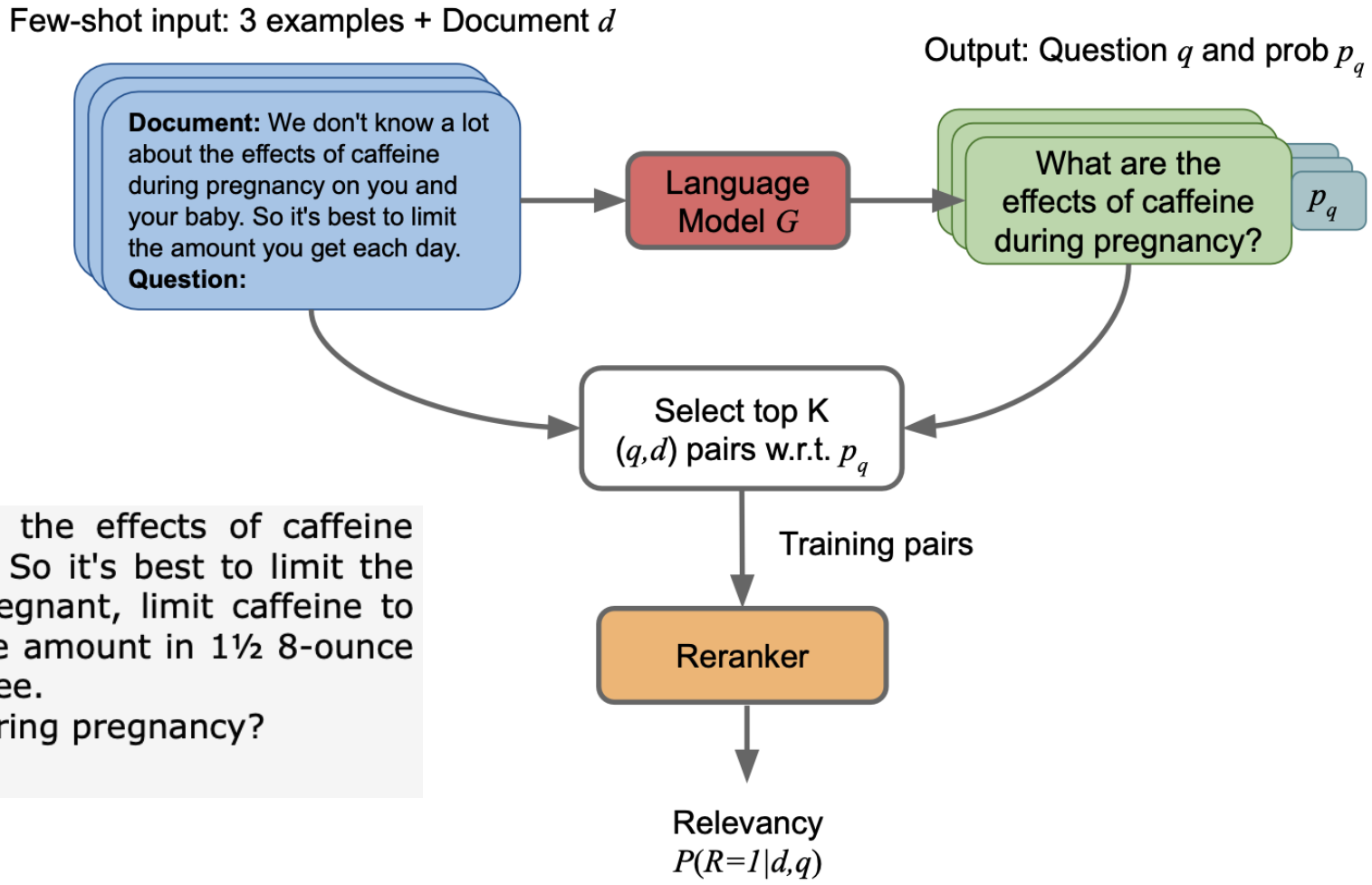


ImageNet



Your small dataset

LLM-based: Dataset Generation

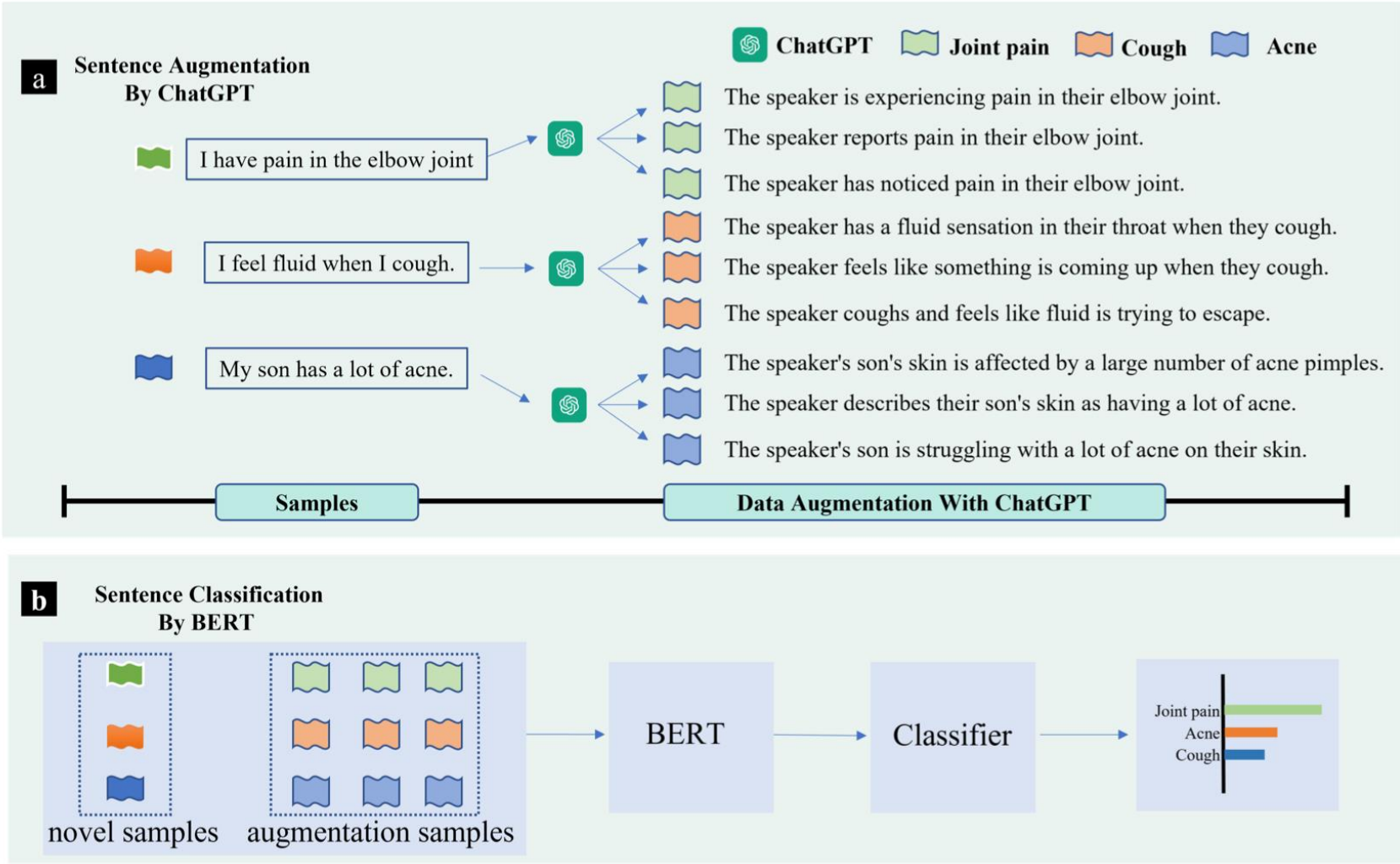


Document: We don't know a lot about the effects of caffeine during pregnancy on you and your baby. So it's best to limit the amount you get each day. If you are pregnant, limit caffeine to 200 milligrams each day. This is about the amount in 1½ 8-ounce cups of coffee or one 12-ounce cup of coffee.
Relevant Query: Is a little caffeine ok during pregnancy?



Ref: InPars: Unsupervised Dataset Generation for Information Retrieval

LLM-based: Text Data Augmentation



Ref: AugGPT: Leveraging ChatGPT for Text Data Augmentation

LLM-based: Generate Triplet samples

You have been assigned a retrieval task: *{task}*
Your mission is to write one text retrieval example for this task in JSON format. The JSON object must contain the following keys:
- **"user_query"**: a string, a random user search query specified by the retrieval task.
- **"positive_document"**: a string, a relevant document for the user query.
- **"hard_negative_document"**: a string, a hard negative document that only appears relevant to the query.

Please adhere to the following guidelines:
- The "user_query" should be *{query_type}*, *{query_length}*, *{clarity}*, and diverse in topic.
- All documents should be at least *{num_words}* words long.
- Both the query and documents should be in *{language}*.

... (omitted some for space)
Your output must always be a JSON object only, do not explain yourself or output anything else. Be creative!

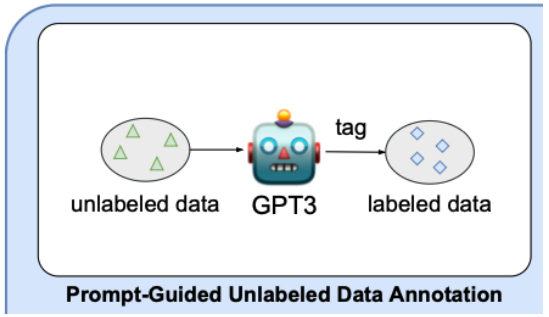
{**"user_query":** "How to use Microsoft Power BI for data analysis",
"positive_document": "Microsoft Power BI is a sophisticated tool that requires time and practice to master. In this tutorial, we'll show you how to navigate Power BI ... *(omitted)* ",
"hard_negative_document": "Excel is an incredibly powerful tool for managing and analyzing large amounts of data. Our tutorial series focuses on how you...*(omitted)*" }

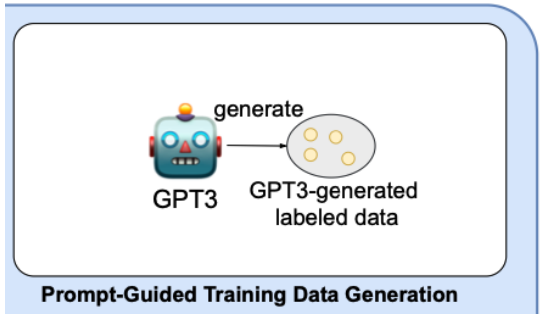
Ref: Improving Text Embeddings with Large Language Models

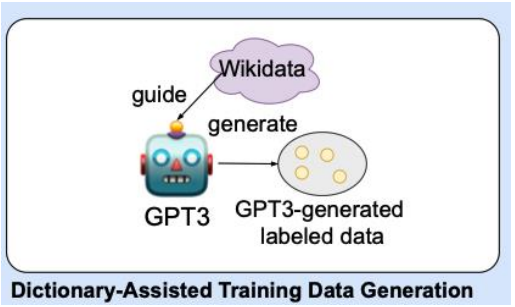
LLM-based: Tagging/generate data

Approach	Num. of Samples	Cost (USD)	Time (Mins)	Results
PGDA	3000	11.31	14†	87.75
	6000	22.63	27†	**89.29**
PGDG	3000	0.91	4†	73.81
	6000	1.83	8†	76.55
DADG	3000	7.18	23†	68.04
	6000	14.37	46†	71.51
Human Labeled	3000	221 - 300	1000	88.47
	67349	4800 - 6700	22740	93.52
PGI	1821	7.33	12	95.77

Table 1: Costs, time spendings and results of SST2. †means multiprocessing (5 processes) is enabled. Time for manual labeling excludes the time spent on instruction preparation and training.

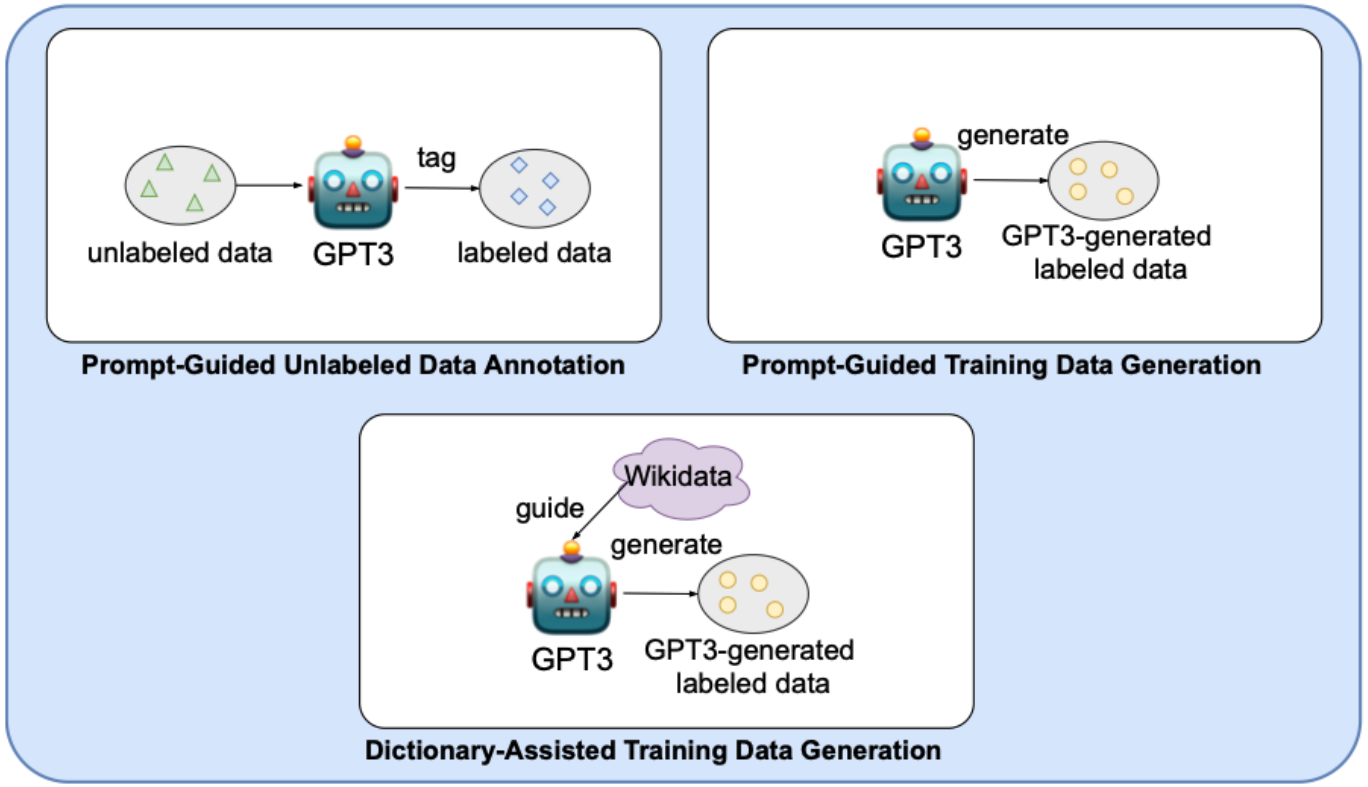






Ref: Is GPT-3 a Good Data Annotator?

LLM-based: Tagging/generate data



Choose the sentiment of the given text from Positive and Negative.
Text: a feast for the eyes
Sentiment: Positive
...
Text: boring and obvious
Sentiment: Negative
Text: [Unlabeled Data]
Sentiment: [Label]

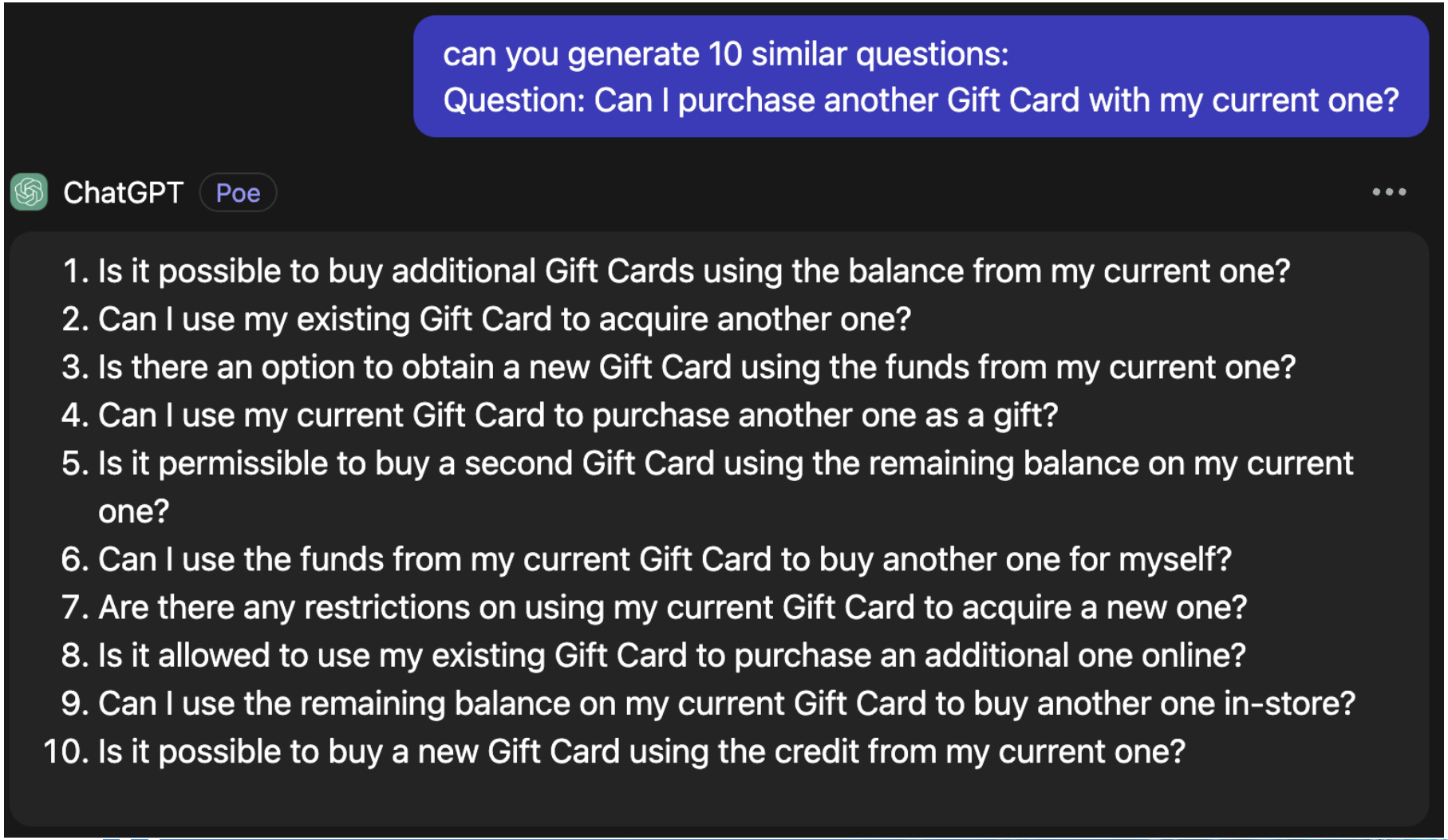
Write 20 different movie reviews with positive sentiments with no more than 20 words.
Sentiment: Positive
Text: a feast for the eyes
...
Sentiment: Positive
Text:

Sentiment: Positive
Text: a feast for the eyes
...
Write a movie review with the given entity with positive sentiment.
Entity: [Entity1]
Sentiment: Positive
Text:



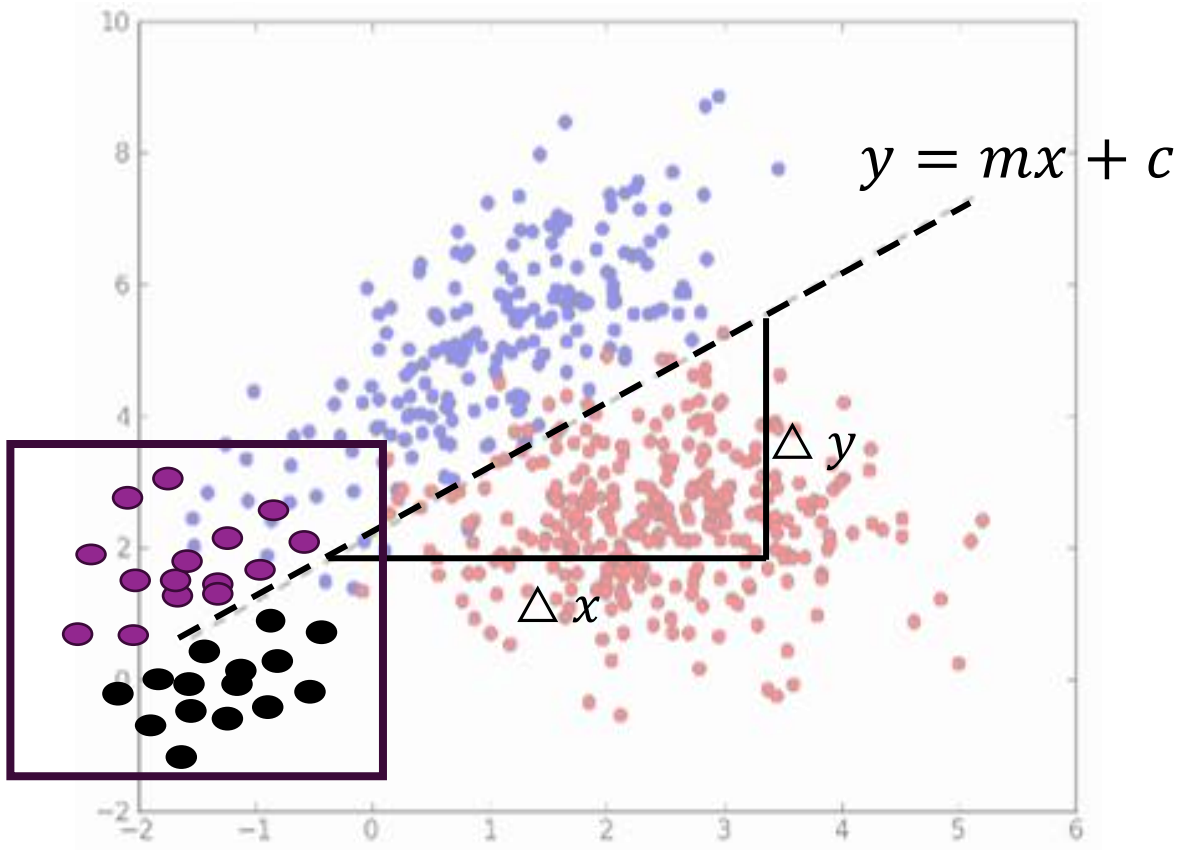
Ref: Is GPT-3 a Good Data Annotator?

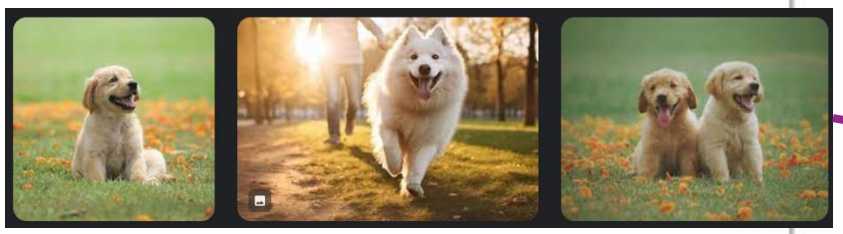
Data Augmentations

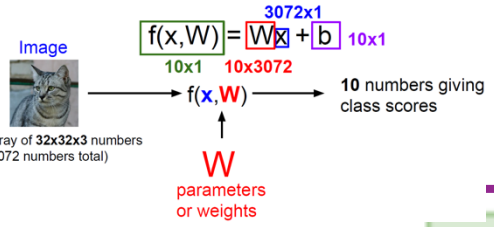


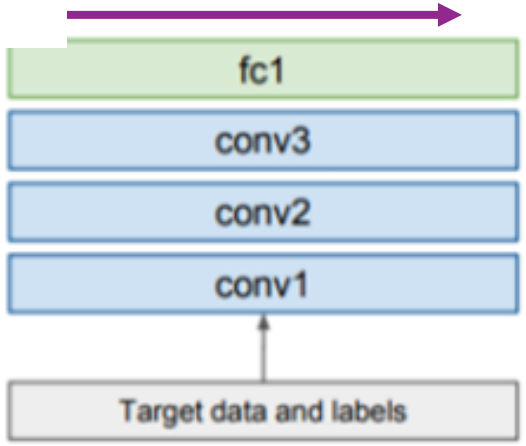
LLM-based: Tagging/generate data

- Adding extra data points



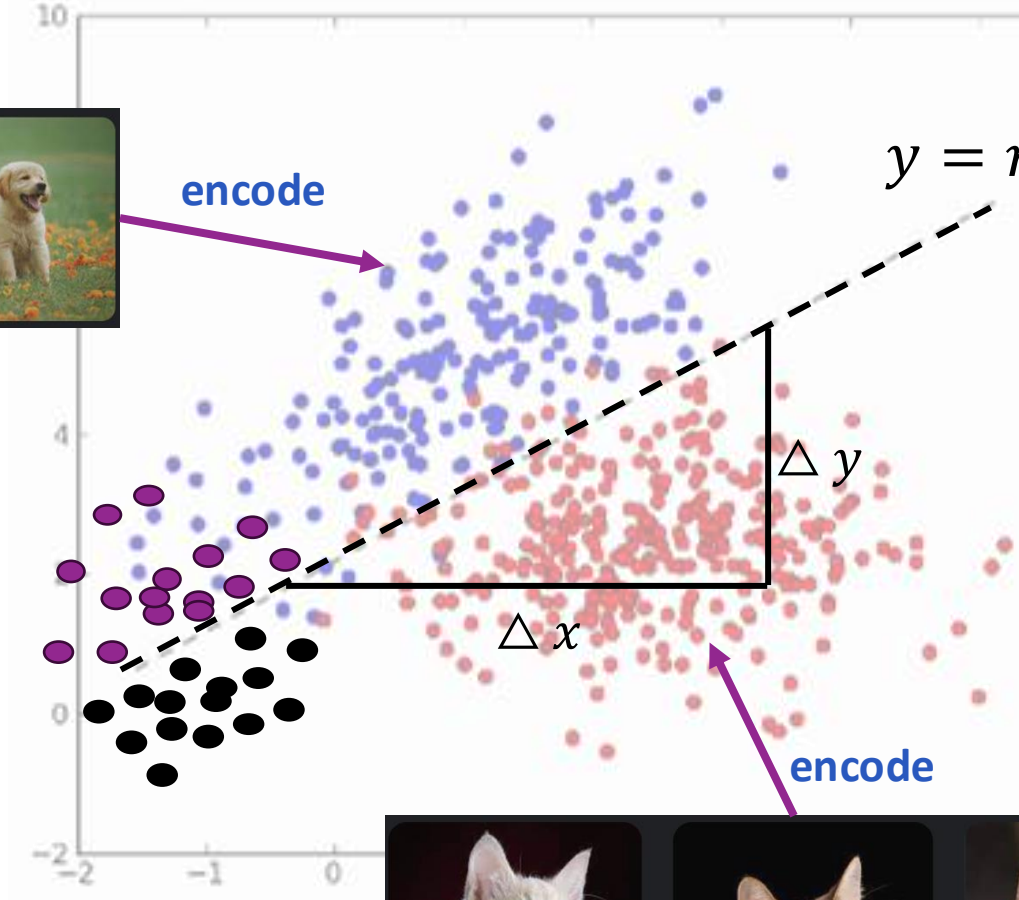






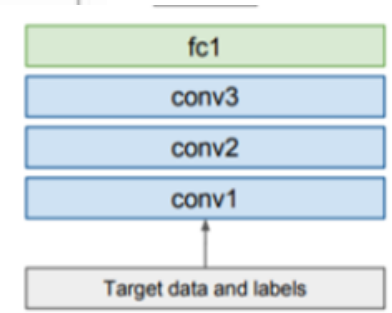
Pre-train model (encoder)

encode

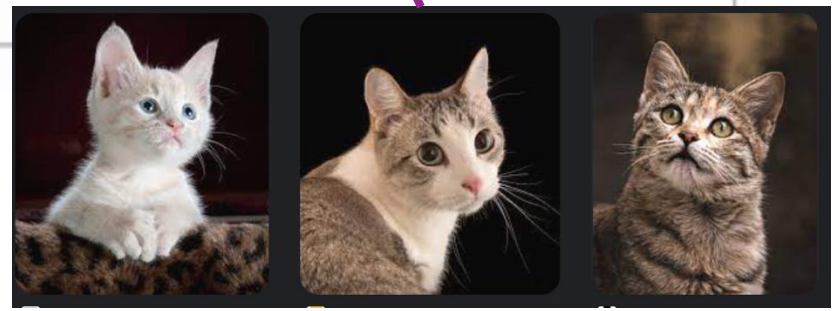


classifier

$$y = mx + c$$



encode





Agenda

- Linear classifier
- Transfer Learning
- Local Large Language Model



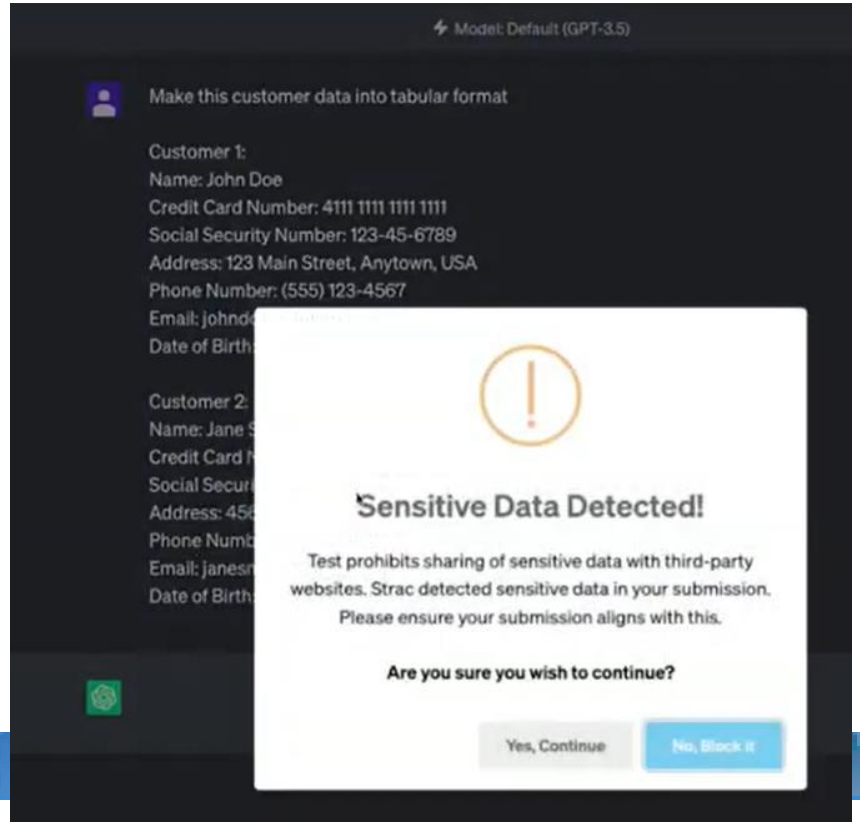
What is Large Language Model

- **What they are:** LLMs are **advanced AI programs** trained on massive datasets of text and code to understand, generate, and translate human-like language.
- **How they work:** Built on **transformer architecture**, they function as powerful prediction machines that analyze context to guess the most mathematically probable next word or token.
- Normally hosted by big tech giants, can called with API



Why Local LLMs?

- **Privacy**: You can prompt local LLMs in a multi-turn manner without your prompt data leaving your localhost.
- **Support and Security**: similar support and security as OpenAI or Claude.
- **Offline Support:** Use LLM while offline.



ref: https://medium.com/@amosgyamfi/the-6-best-llm-tools-to-run-models-locally-eedd0f7c2bbd

Why Use Local LLMs?

- **Customization Options**: Local LLMs provide advanced configurations for CPU threads, temperature, context length, GPU settings, and more.

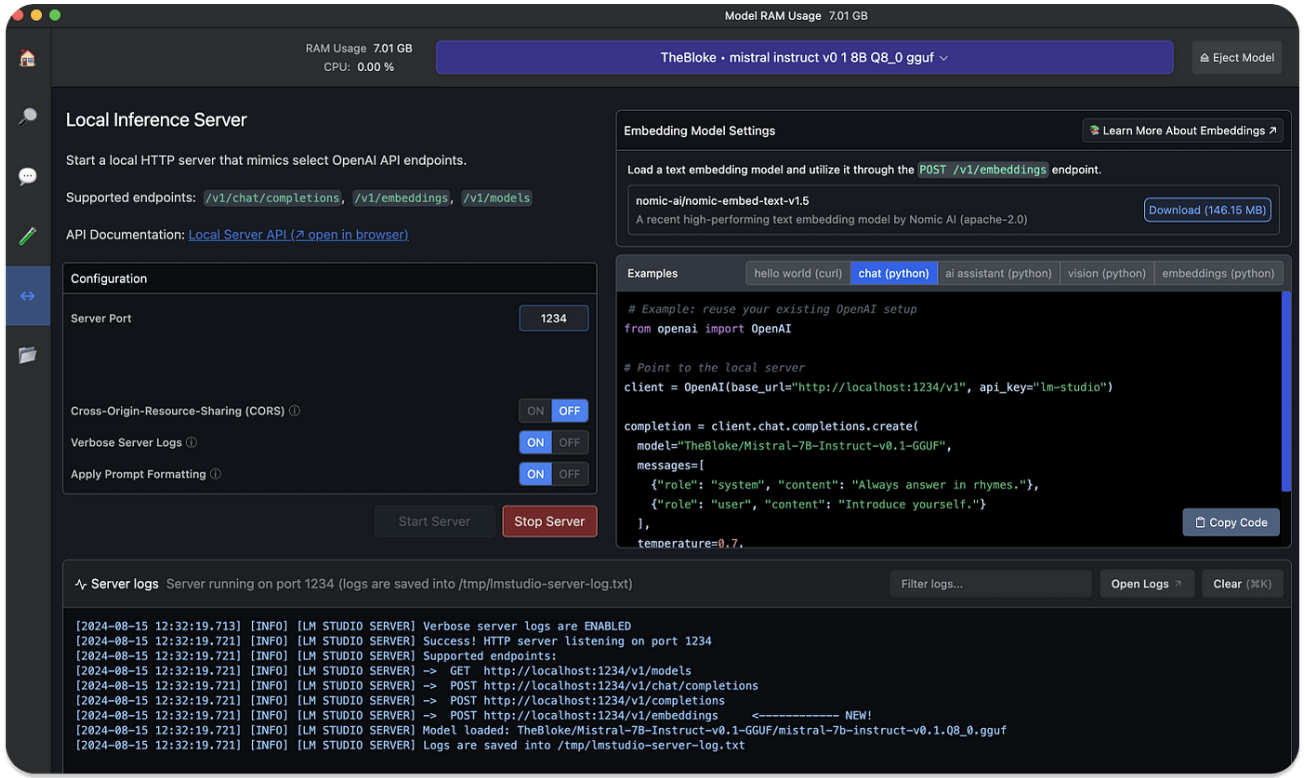


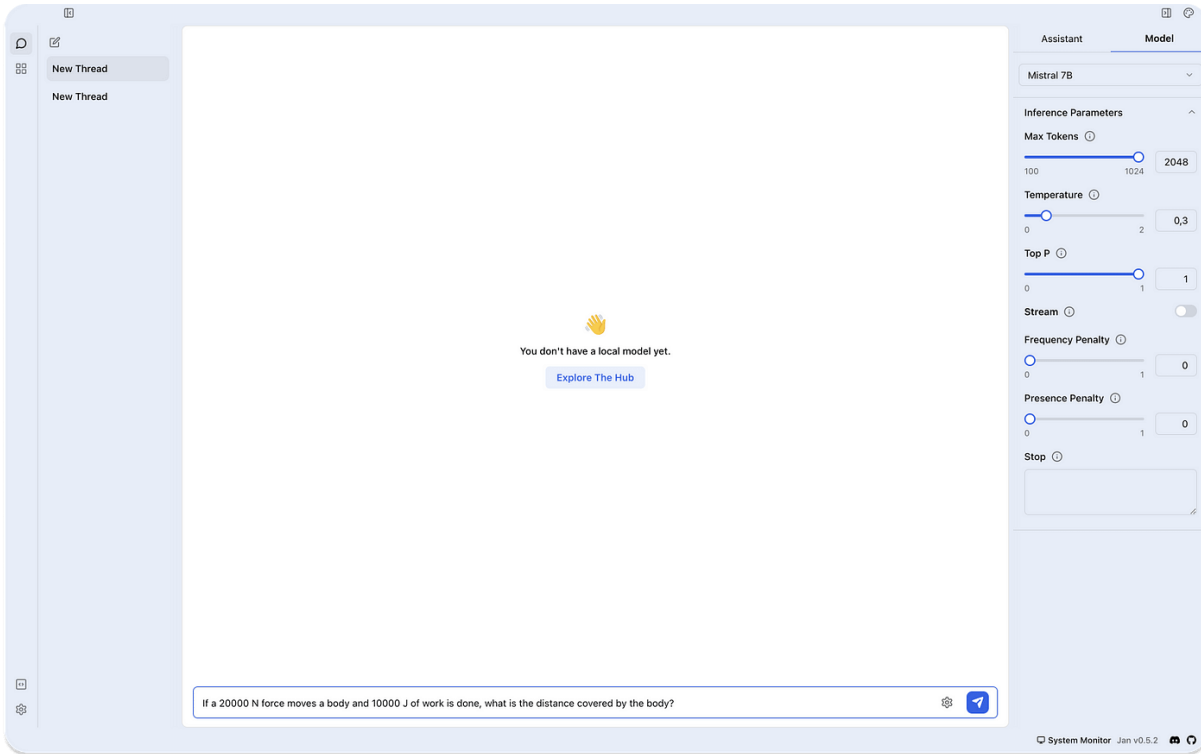
- **Subscription and Cost**: These tools are free to use a to save money

ref: https://medium.com/@amosgyamfi/the-6-best-llm-tools-to-run-models-locally-eedd0f7c2bbd

How to Use Local LLMs?

- software-based: Jan/Llamafile / LM Studio and more





How to Use Local LLMs?

Hugging Face

- Code-based:

Example Code

Here's a simple example of how to generate text using GPT-2:

python Copy

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Load pre-trained model and tokenizer
model_name = 'gpt2'  # You can also use 'gpt2-medium', 'gpt2-large', etc.
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)

# Input text prompt
prompt = "Once upon a time in a land far away"

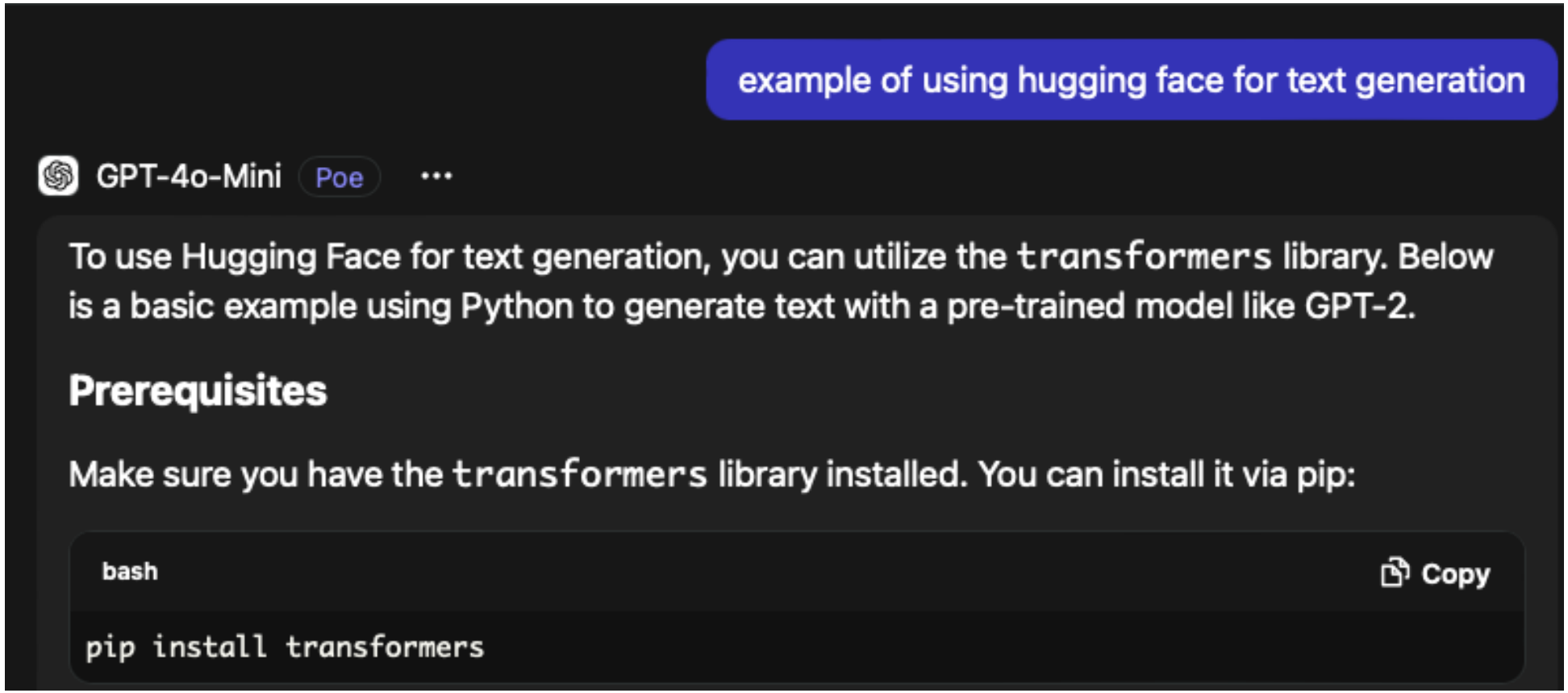
# Encode the prompt
input_ids = tokenizer.encode(prompt, return_tensors='pt')

# Generate text
output = model.generate(input_ids, max_length=50, num_return_sequences=1)

# Decode the generated text
generated_text = tokenizer.decode(output[0], skip_special_tokens=True)

print(generated_text)
```

How to use HuggingFace with Transformers (Local Models)



Other options: Ollama

Python

```python
import ollama

def generate_response():
    try:
        response = ollama.chat(
            model='llama3',  # Replace with 'mistral', 'phi3', or whatever model you have
            messages=[
                {
                    'role': 'user',
                    'content': 'Why is the sky blue? Give me a short, one-sentence answer
                },
            ]
        )
        # Access the response content
        print("Response:")
        print(response['message']['content'])

    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    generate_response()
```



Compute & Memory Requirements

- Memory Required (GB) = (Number of Parameters × Quantization Bits ÷ 8) + 2 GB (allocated for Context Window/KV Cache)
- Compute needs scale with parameter size
- FLOPs per token: ≈ 2P (P = parameters)
- 7B models:~14B FLOPs/token
- 70B models:~140B FLOPs/token



Model Size	Common Models (4-bit Quantized)	Minimum VRAM	Recommended Hardware Setup	Performance Expectations
<4B	• Gemma 4 (E2B / E4B) • Qwen 3 (0.6B / 1.7B) • Phi-4 Mini (3.8B) • SmolLM2 (1.7B)	**4 GB – 6 GB**	• Intel/AMD Integrated Graphics • NVIDIA GTX 1060 / RTX 3050 • Base Apple M-series Mac (8GB RAM)	Ultra-fast text generation and lightweight tasks. Easily runs on basic laptops, modern handhelds, and edge devices with native audio/image capabilities.
7B- 9B	• Llama 4 Scout (11B) • Qwen 3 (8B / 14B) • Phi-4 Reasoning-Vision (15B) • Gemma 3 (12B)	**6 GB – 10 GB**	• NVIDIA RTX 3060 (12GB) / 4060 • AMD Radeon RX 6700 • Apple Mac with 16GB+ Unified Memory	Exceptional speed and intelligence ratio. Serves as the sweet spot for consumer gaming hardware, handling complex instructions, agentic tool use, and multimodal processing.



Model Size	Common Models (4-bit Quantized)	Minimum VRAM	Recommended Hardware Setup	Performance Expectations
14B - 32B	• Gemma 4 (26B A4B MoE / 31B Dense) • Qwen 3.8 (27B) • Qwen 3 (32B) • Command R (35B)	**12 GB – 24 GB**	• NVIDIA RTX 4070 Ti Super (16GB) • NVIDIA RTX 3090 / 4090 (24GB) • Apple Mac with 24GB or 36GB Memory	Near-instantaneous responses on single 24GB GPUs. Provides deep mathematical reasoning, vision-language context, advanced coding capabilities, and dynamic thinking modes.
70B+	• DeepSeek-V4-Flash (284B MoE / ~13B Active) • DeepSeek-V4-Pro (1.6T MoE / ~49B Active) • Kimi K3 (2.8T MoE) • Llama 4 Maverick (MoE) / Qwen 3.8-Max	**40 GB – 128 GB+ (GGUF/quantized MoE offloading)**	• Dual NVIDIA RTX 3090 / 4090 (48GB) • Workstation Rig (1× A100/H100 or RTX 6000 Ada) • Mac Studio / Pro (64GB to 192GB+ Unified Memory)	Frontier-class reasoning, million-token context windows, and commercial-grade tool-use capabilities. Requires multi-GPU setups, workstation cards, or high-capacity Apple Silicon.



Latency vs Throughput

- Smaller models: Lower latency, suitable for real-time tasks
- Larger models: Higher throughput, better for batch processing
- Mistral's SWA and DeepSeek's MLA improve efficiency

Table: Latency vs Throughput Suitability

Model	Low Latency	High Throughput
Llama-7B/8B	Good	Moderate
Mistral-7B	Excellent	Moderate
DeepSeek-7B	Excellent	Moderate
Llama-70B	Moderate	Excellent
DeepSeek-67B	Moderate	Excellent



GPT-OSS (20B): OpenAI's Reasoning Powerhouse

- **Strengths:**
  - Exceptional on complex reasoning, math, and multi-step logic.
  - Outperforms other open models of similar size on challenging benchmarks like AIME.
  - Can use tools: code execution, web browsing, and document search.
  - Open license allows fine-tuning and commercial use.
- **Weaknesses:**
  - Less reliable for trivia, pop culture, and basic fact recall.
  - Not the strongest pure coding model better at reasoning about code than generating it flawlessly.



Gemma 4 (E2B / E4B / 12B / 26B / 31B) — The Open Agentic Powerhouse

- **Strengths:**
  - Native Multimodality: Offers built-in text, image, video, and native audio support (12B and smaller).
  - Encoder-Free Architecture: The 12B variant feeds vision/audio directly into the LLM loop, saving memory and latency.
  - Agentic Reasoning: Native <|think|> loop triggers massive gains in multi-step planning, coding, and math.
  - Fast Local Speeds: Ships with Multi-Token Prediction (MTP) drafters out of the box for accelerated local inference.
- **Weaknesses:**
  - Overly Strict Guardrails: Highly prone to false-positive refusals on harmless or ambiguous edge-case prompts.
  - Modality Sensitivity: Requires strict, rigid prompt formatting (e.g., visuals before text) to prevent instruction drift.
  - High MoE Memory Footprint: The 26B Mixture-of-Experts variant requires loading all 25B+ parameters into memory despite only routing ~3.8B per token



Qwen 3.6 (27B / 35B-A3B) — Next-Gen Agent & Thinking Powerhouse

- **Strengths:**
  - Agentic Coding Leader: Heavily optimized for repository-level reasoning and front-end development workflows, excelling at complex coding challenges (e.g., scoring highly on SWE-bench Verified).
  - Thinking Preservation: Features a dedicated mechanism to retain and pass reasoning/chain-of-thought context seamlessly across multi-turn historical messages, making iterative debugging and development highly efficient.
  - Efficient MoE Architecture: The 35B-A3B variant utilizes a sparse Mixture-of-Experts routing system, enabling lightning-fast inference throughput and minimal compute overhead compared to traditional dense models.
  - Massive Context & Multilingual Reach: Native support for an expanded 256K context window and comprehensive coverage across 201 distinct languages and regional dialects.
- **Weaknesses:**
  - Thinking Overhead: Preserving detailed internal chain-of-thought context over massive chat histories requires careful memory management, which can increase VRAM consumption during long development sessions.
  - Architecture Complexity: Transitioning between dense (27B) and MoE architectures (35B-A3B) means users must optimize separate local quantization strategies (like Unsloth Dynamic GGUFs) to maintain peek local performance.



Qwen3-VL (8B / 32B / 235B-A22B) — Advanced Multimodal & Spatial Vision Leader

- **Strengths:**
  - Visual Reasoning: Features "Thinking" editions that use internal step-by-step reasoning and dynamic image-zooming tools to analyze fine-grained details.
  - Spatial 3D Grounding: Supports precise 3D object bounding for indoor and outdoor environments, making it ideal for robotics and embodied AI.
  - GUI Agent Automation: Optimized for desktop and mobile screen navigation, accurately mapping coordinates to execute autonomous workflows.
  - Massive Video Context: Handles a 256K context window (expandable to 1M) for interleaved text, images, and long-horizon video tracking.
  - Advanced OCR & Math: Uses deep feature fusion to deliver top-tier multi-language text extraction and visual-math reasoning.
- **Weaknesses:**
  - High Compute Demands: The flagship 235B MoE variant requires massive GPU infrastructure and specialized quantization to run locally.
  - VRAM Overhead: Real-time tracking of long, second-level video metadata rapidly consumes memory during long-sequence inference.



How do you select a model?

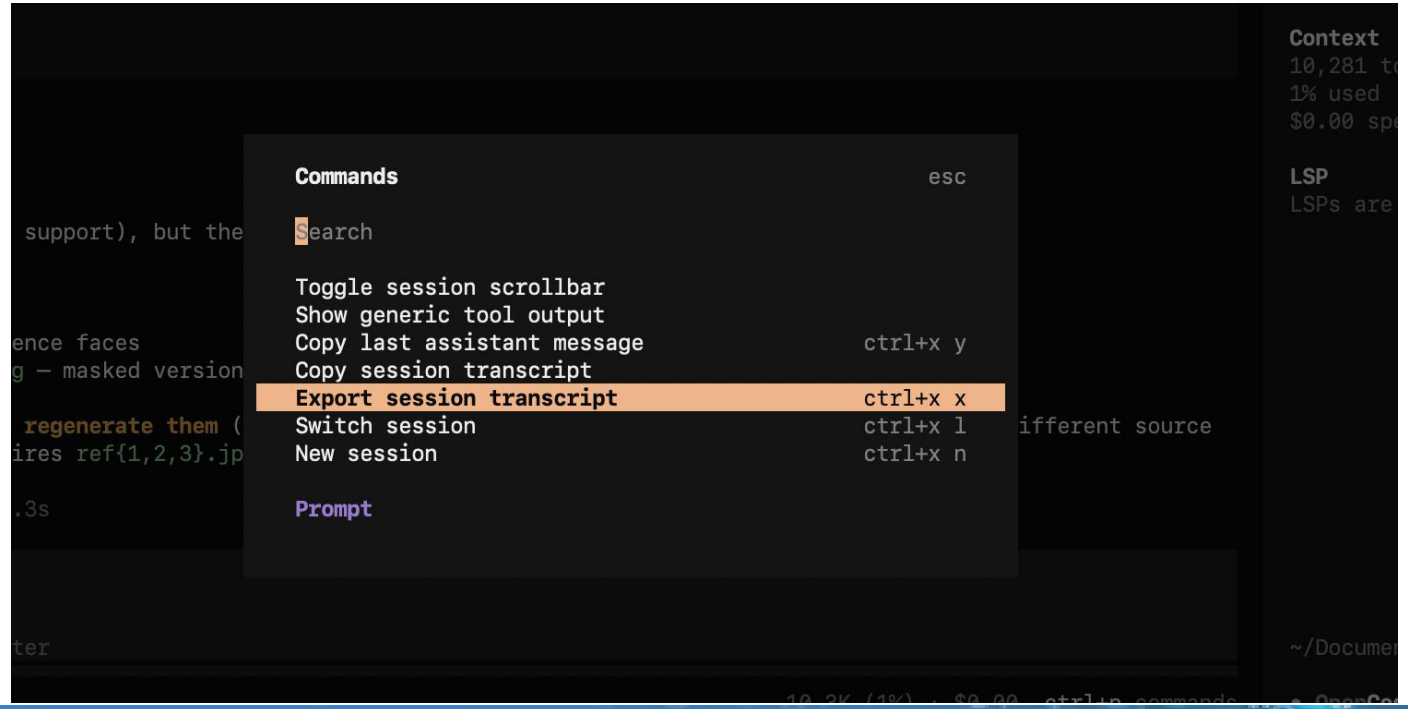
Model	Features: Context Window	Features: Creator	Intelligence: Artificial Analysis Intelligence Index	Price: Cost per Task USD	Speed: Median Tokens/s	Latency: Latency First Chunk (s)	End-to-End Response Time: Total Response (s)	Further Analysis
Claude Fable 5.1 (max with fallback)	1M	Anthropic	53	$7.63	67	234.98	242.43	Model ↗ | Providers ↗
Claude Fable 5.1 (xhigh with fallback)	1M	Anthropic	53	$5.98	62	85.94	93.96	Model ↗ | Providers ↗
GPT-6 Astra (max)	1M	OpenAI	53	$3.26	60	320.98	329.26	Model ↗ | Providers ↗
GPT-6 Astra (xhigh)	1M	OpenAI	53	$2.31	55	128.37	137.44	Model ↗ | Providers ↗
Claude Fable 5.1 (high with fallback)	1M	Anthropic	51	$3.91	57	16.39	25.11	Model ↗ | Providers ↗
GPT-6 Astra (high)	1M	OpenAI	51	$1.72	52	33.69	43.22	Model ↗ | Providers ↗
Claude Opus 5 (max)	1M	Anthropic	51	$5.86	53	52.98	62.47	Model ↗ | Providers ↗
Claude Fable 5 (with fallback)	1M	Anthropic	50	$8.75	65	81.62	89.28	Model ↗ | Providers ↗
GPT-6 Astra (medium)	1M	OpenAI	50	$1.54	52	4.35	14.04	Model ↗ | Providers ↗
Claude Opus 5 (xhigh)	1M	Anthropic	50	$4.88	54	28.65	37.98	Model ↗ | Providers ↗



ref: https://artificialanalysis.ai/leaderboards/models

Demo

- Using Ollama
- Calling LLM with Ollama with Python
- Output the full history as part of the grading





Tasks: Build an image classification using OpenCode

- Basic Tasks (50%)
  1. Download a set of images and encode them using pre-trained ResNet model
     - At least 100 cat images and 100 dog images
  2. Find and visualize the top 5 most similar and dissimilar images, for a given query image
  3. Build a KNN classifier using the dataset
  4. Visualize the performance using 10 unseen images, visualize failed cases
  5. Build a website to show the results
- Advance (50%)
  1. Implement and compare 3 different encoding models
  2. Implement and compare 3 different distance calculations in L1
  3. Analyze their strength and weakness using the results, and explain using the knowledge from STAT6207
- Expected outcomes and notes:
  - Submit your works to blackboard by Wednesday (23 Sep 26)
  - Make a report in pdf to explain your results and the decision while using OpenCode
  - Submit your session transcript, .py, pdf, the website and etc.



Questions?

Email: pakmingcheung@cuhk.edu.hk

